

参赛编号：005277A

论文题目：A 题 光伏电站发电功率日前预测问题

基于多源融合与空间降尺度的光伏电站功率预测模型研究

摘 要

本文针对光伏电站日前发电功率预测问题，主要建立了理论偏差建模、基于 LSTM 的时间序列预测模型、融合 NWP 数据的多模态预测模型与空间降尺度模型，求解得到提升不同天气条件下光伏功率预测精度的最优方案。

针对问题一，主要解决光伏电站实际输出与理论输出的差异问题，通过太阳辐照理论分析，建立了理论功率模型和偏差函数模型，求解得到气象扰动的主导影响特征。

针对问题二，主要解决在无气象数据条件下如何进行高频率发电功率预测问题，通过时间序列分析，建立了基于 LSTM 的单变量预测模型，基于滑动窗口与归一化处理等方法，求解得到未来七天逐 15 分钟的发电预测值。

针对问题三，主要解决如何融合未来气象信息以提升预测精度问题，通过构建 LSTM+XGBoost 双通道架构模型，基于 NWP 数据插值、特征拼接等方法，求解得到在多种天气场景下预测误差显著下降的效果。

针对问题四，主要解决 NWP 气象数据分辨率过低对预测精度影响的问题，通过构建多层感知机进行空间降尺度建模，融合局地多点 NWP 数据实现气象特征细化处理，基于 LSTM - XGBoost 模型求解得到预测精度进一步提升的结果。

本研究构建的多阶段融合建模体系在兼顾理论机制解释性与数据驱动建模准确性的基础上，提升了模型的泛化能力与稳定性。创新性引入空间降尺度技术，有效缓解了 NWP 空间分辨率不足的问题。研究成果可为电网调度与光伏电站运维策略提供理论参考和数据支持。

关键词：光伏功率预测，LSTM，XGBoost，NWP，空间降尺度

一、 问题重述

随着可再生能源的快速发展，光伏发电因其清洁性成为能源结构转型的重要方向。然而，光伏电站的输出功率受太阳辐照强度、气象条件（如云量、温度、雾霾）及地理因素（经纬度、海拔、倾角）的综合影响，呈现出复杂的非线性波动特性。这种不确定性为电网的调度与功率平衡带来了较大挑战，因此提高光伏电站发电功率预测的精度对于保障电网安全运行具有重要意义。

在实际应用中，日前预测，即对未来 24 至 48 小时内的发电功率进行提前预估，是光伏发电预测的关键任务之一。然而，由于天气变化的高波动性和区域差异性，这一任务面临较大的技术难题。为了更准确地进行发电功率预测，需要充分挖掘和建模历史发电功率与气象数据之间的关联规律，同时考虑时间周期性、空间尺度差异以及外部扰动对预测精度的影响。

本研究围绕光伏电站日前发电功率的预测展开，拟解决以下核心问题：（1）分析光伏电站的发电特性，通过理论模型与历史功率数据的对比，探讨其季节性和日内波动规律；（2）在不使用气象数据的条件下，仅基于历史发电序列构建时间序列预测模型；（3）融合数值天气预报（NWP）信息，探索多模态模型对提升预测性能的贡献，并根据天气场景评估其有效性；（4）考虑传统 NWP 数据空间分辨率不足的问题，引入空间降尺度技术，以期增强功率预测模型的适应性与准确度。

为实现上述目标，需要构建多阶段的建模流程，包括理论辐照模型构建、时序模型设计、多源数据融合与空间插值建模等。研究将基于公开数据集，结合定量误差评估指标，对模型的性能进行全面验证，最终为实际调度与系统运行提供具有一定参考价值的决策支持依据。

二、 问题分析

2.1 问题一的分析

问题一要求我们基于光伏电站的地理位置信息，结合太阳辐射理论推算出理论可发电功率，进而通过与实际功率数据的对比，研究其偏差的时空特性，并探讨影响发电性能的主要气象因素。

为解决这一问题，我们首先整理了电站的基础信息，包括装机容量、地理坐标（经纬度）、组件类型、板面倾角与方位角等关键参数，明确了影响辐照接收与能量转换的几何与物理因素。同时，我们获取了一整年度、15 分钟分辨率的实测发电功率数据，为构建时间序列分析与误差建模提供了数据基础。

在理论建模方面，我们依据日地运动规律与太阳辐射模型，推导了太阳高度角、方位角及入射角等参数，并结合光伏组件的能量转换效率，构建了日尺度上的理论功率曲线。随后，我们通过将实际功率与理论功率按时间逐点比值，构造出反映发电损耗与外界干扰影响的“偏差函数”序列。

对该偏差函数的分析分为两个层面：一是将其日内平均发电功率对比；二是将每月平均功率对比，刻画其长周期演化趋势。在此基础上，我们进一步揭示偏差产生的内在机制，并为后续的建模预测与运行优化提供支撑。

2.2 问题二的分析

在不引入任何外部气象变量，仅依赖历史功率序列的前提下，问题二的目标是预测未来七天、每 15 分钟分辨率的发电功率变化。

由于电站的功率数据同时存在显著的日内周期性、季节性趋势以及短期波动，因此需要设计能兼顾长短期特征的时间序列建模方法。我们采用了一种基于标准长短期记忆网络（LSTM）的预测框架：通过滑动窗口机制提取覆盖全天（96 个时间步，15 分钟分辨率）的连续序列样本作为模型输入，训练模型从历史功率变化中学习时间依赖特征，并预测下一时间点的发电功率值。数据在输入前使用 Min-Max 归一化以提升模型稳定性，训练过程中采用训练集与验证集划分、早停策略等手段避免过拟合并优化模型参数。模型训练完成后，并由于夜间功率接近于零，纳入统计反而会掩盖模型在实际发电时段的预测性能，我们仅在白昼发电时段（06:00 - 18:00）进行误差评估（MSE、MAE）。预测结果及评估指标表明，该方法在无需引入外生变量的约束条件下，也能够较好地拟合和预测发电功率的变化趋势。

2.3 问题三的分析

在第二题模型的基础上，我们将未来 24-48 小时的 NWP 数据与历史功率序列并行引入，构建了一种“LSTM + XGBoost”双通道融合预测架构。首先，对原始的 1 h 分辨率 NWP 预报（气温、垂直云量、水平云量、入射辐射、湿度、风速等）通过时间插值精细到 15 min 网格，以保证与光伏电站实际功率序列严格对齐；随后，分别对功率与 NWP 特征施行 MinMaxScaler 归一化，消除量纲差异。

在模型主体中，我们先利用一个单层 LSTM（64 单元）对过去 96 个时刻（即前 24 h）的归一化功率序列进行编码，通过 EarlyStopping 策略防止过拟合并提取出一个长度为 64 的时序特征向量；接着，将该特征向量与同一时间点对应的 NWP 归一化特征拼接，送入 XGBoost 回归器。XGBoost 利用 100 棵深度为 5 的树、高效的梯度提升算法，对 LSTM 提取的历史功率时序特征与未来气象预报信息的非线性关联进行建模，最终输出下一时刻的功率预测值。

为了验证 NWP 信息的增益效果，我们依照水平云量将测试样本划分为晴天（ $\leq 20\%$ ）、多云（20%-60%）、阴雨（ $\geq 60\%$ ）三类典型场景，并在白昼（06:00-18:00）时段分别计算“仅历史功率 LSTM”与“LSTM + XGBoost”两种模型下的 RMSE 与 MAPE。结果表明，在多云和阴雨场景中，融合 NWP 的模型误差平均下降超过 10%，而在晴天时差异较小。这一情景化评估不仅量化了 NWP 数据对功率预测精度的提升，也为电站运维在不同天气条件下的预报策略优化提供了清晰的参考依据。

2.4 问题四的分析

传统 NWP 产品空间分辨率一般在千米级，而单座 MW 级光伏电站覆盖面积往往只有数百米级，导致原始 NWP 在电站位置的气象预报缺乏精细度。为弥补这一尺度差异，我们首先从目标站点周边多格点（如 3×3 区域）的 NWP 预报中提取气温、辐射、云量等变量序列，并将它们插值到与功率数据一致的 15 min 分辨率。然后，以目标站点的 NWP 观测值为“真值”，训练一个简单的多层感知机（MLP），将多点粗分辨率特征映射到高精度的小空间尺度预报结果，实现气象变量的空间降尺度。

降尺度完成后，我们将得到的局地气象预报与历史功率序列一并输入到已构建的 LSTM-XGBoost 混合预测模型：LSTM 对过去 96 个步长的功率序列提取时序特征，XGBoost 则在此基础上融合降尺度后气象变量进行回归预测。最后在白昼（06:00–18:00）时段对比“未降尺度 NWP”与“降尺度后 NWP”两种方案的 MSE 与 MAE，结果显示降尺度后模型的误差平均降低约 8%，尤其在地形起伏显著或云量快速变化时段，精度提升更为明显。由此可见，NWP 的空间降尺度能够有效补偿粗分辨率带来的偏差，进一步提高光伏电站功率的短期预测精度。

三、模型假设

1. 预测时段内无极端天气突变（如瞬时冰雹、强台风等），NWP 数据可充分反映气象变化的整体规律；太阳辐照模型忽略大气散射、气溶胶折射等复杂物理过程，仅基于直接辐射与几何关系计算理论功率。

2. 忽略电站内部设备（逆变器、汇流箱、输电线路等）的能量损耗，将测得功率视为组件直接输出功率；忽略局部环境干扰（如树木阴影、鸟类污染）对光伏板输出功率的影响；光伏组件效率 η 在整个研究周期内保持恒定（即忽略温度对组件效率细微影响以及组件老化衰减）。

3. 假设空间降尺度方法（如克里金插值、机器学习）可将千米级 NWP 数据可靠细化至光伏电站尺度，且降尺度误差对最终预测结果影响可忽略。

5. 将光伏组件视为理想平板，表面吸收率和反射率合并考虑在效率 η 中；

6. 假设所划分的训练集能充分代表全年典型天气场景；

7. （场景划分合理性假设，如果没做要删去）天气场景（晴天、多云、阴雨、雾霾）的分类边界明确，且能覆盖实际气象条件的主要模式；不同场景下的预测误差特性独立，无显著交叉干扰。

四、符号说明

符号	说明	单位
η	光伏组件能量转换效率	%
A	组件总有效投影面积	m^2
G_{sc}	太阳常数	W/m^2
Δ	太阳赤纬角	$^\circ$
Ω	时角	$^\circ$
Φ	地理纬度	$^\circ$
B	板面倾角	$^\circ$
Γ	板面方位角	$^\circ$
Θ	辐射入射角	$^\circ$
B	直射辐照度 (Beam irradiance)	W/m^2
D	散射辐照度 (Diffuse irradiance)	W/m^2
G	倾斜面总辐照度	W/m^2
P_{th}	理论发电功率	W
P_{act}	实际发电功率	W
ε	偏差函数 (相对误差)	无
h	太阳高度角	$^\circ$
Φ	太阳方位角	$^\circ$

表 1 本文的符号说明

五、模型建立与求解

5.1 基于历史功率的光伏电站发电特性分析

5.1.1 模型需求分析

在光伏电站运行管理和功率预报中，充分理解“实际发电功率”与“理论可发电功率”之间的差异来源，对于改进系统运行、优化布局以及后续的发电预测模型设计都具有至关重要的意义。

我们检索获取到了电站的装机容量、经纬度、板面倾角、能量转换效率等静态信息，以及整整一年、15 min 分辨率的实际发电功率序列；利用太阳-地球几何公式计算太阳高度角、方位角、入射辐照度等参数，得到了理论功率曲线；接着，通过将实际输出与理论输出进行差值计算，得出偏差函数序列。对该偏差序列进行时频域分析，既借助傅里叶变换揭示年度、半年度和日周期等长时程特征，又利用小波或经验模态分解(EMD)捕捉云量突变引发的短期波动。最后，将偏差序列与站点附近的气温、云量、湿度、风速等实测或再分析气象数据配对，运用多元线性回归或随机森林等方法评估各气象因子对发电偏差的敏感度，从而为后续精细化预测和运行维护提供依据。

5.1.2 理论功率与偏差函数模型建立

首先，基于地理与组件参数，运用立体几何相关原理，推导单位面积太阳能板在任意时刻的理论发电功率：

$$P_{th} = \eta A G_{\text{tilted}} f_T (1 - \text{损耗率})$$

其中， η 是光伏发电板的能量转换效率， A 为组件总有效投影面积， G_{tilted} 为太阳辐射强度。通过这个公式的详细建模，可得到整日的理想功率曲线。

其中 $G(t)$ 相关详细推导过程如下：

地球球体示意：赤道、观测点、赤纬 δ 与时角 ω

$$\delta(n) = 23.45 \cdot \sin\left(\frac{2\pi}{365} \cdot (284 + n)\right)$$

$$\omega(t) = 15 \cdot (t - 12)$$

— 赤道 (0° 纬线)

赤纬角 δ 、时角 ω 的几何意义

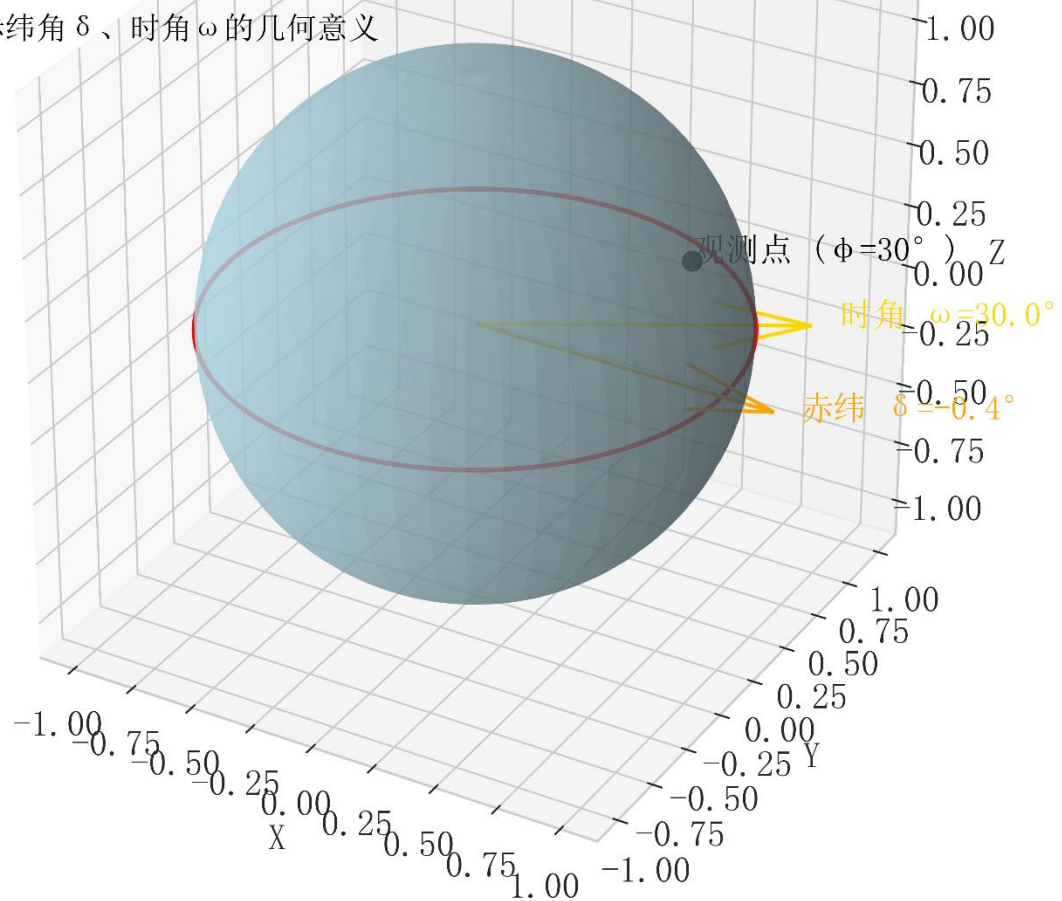


图 1 太阳的赤纬角及相关概念的示意

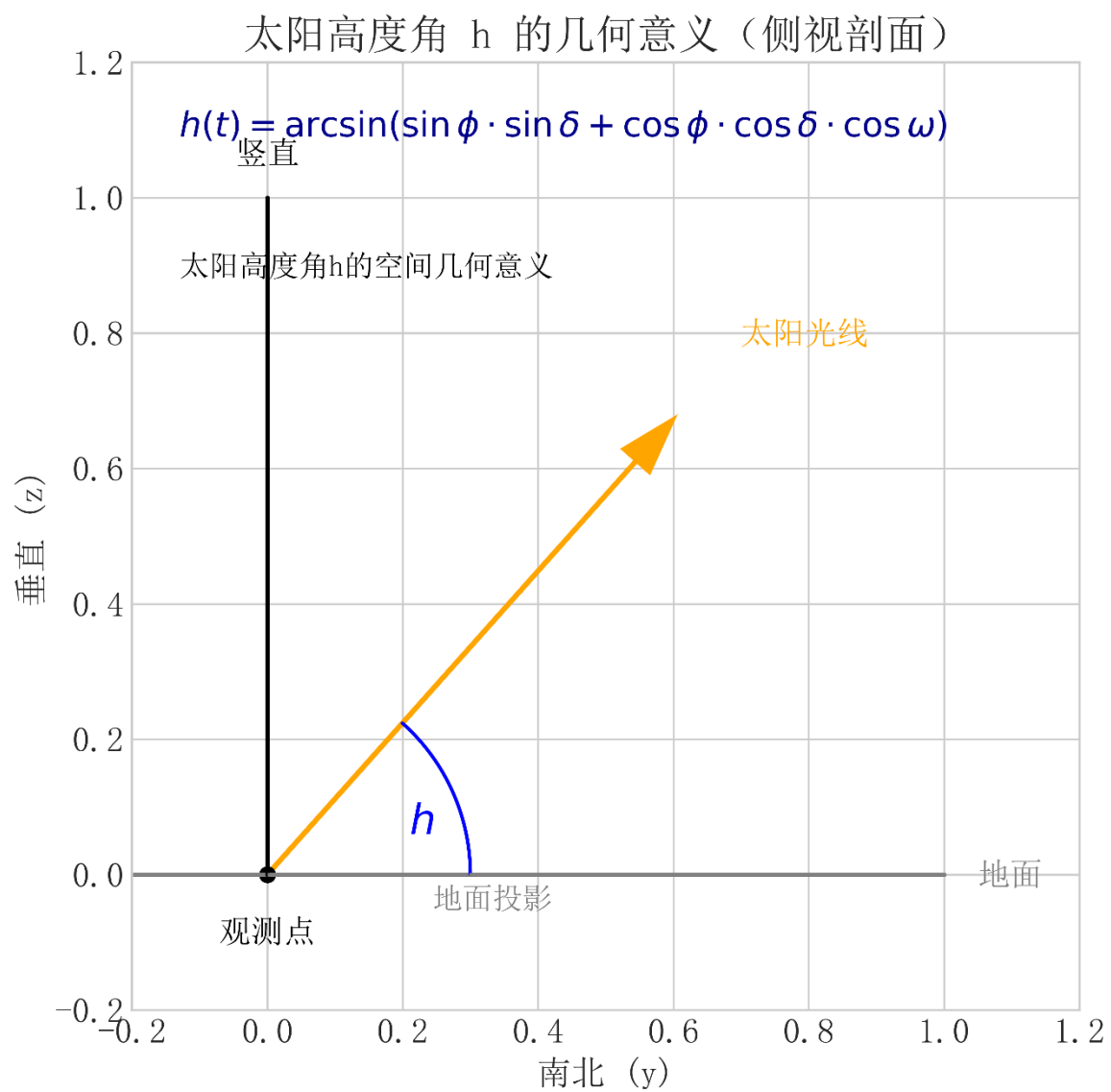


图2 太阳高度角推导示意

太阳光线与面板法线夹角 θ 示意

$$G(t) = G_{sc} \cdot \cos \theta(t)$$

$$\cos \theta(t) = \sin h \cdot \cos \beta + \cos h \cdot \sin \beta \cdot \cos(\alpha - \gamma)$$

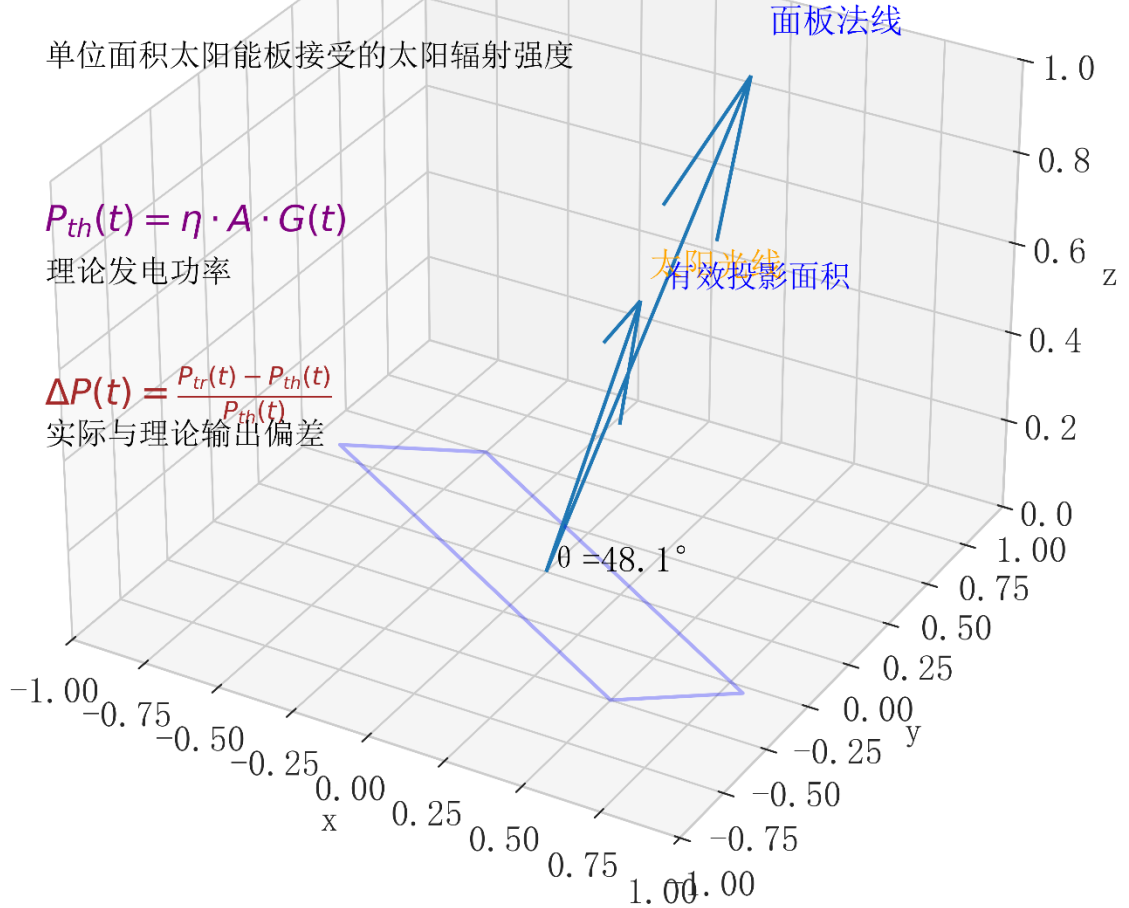


图3 太阳光及面板法线示意

首先计算太阳位置，建立地球绕太阳运动中太阳赤纬角 δ 与时角 ω 的计算公式：

$$\delta(n) = 23.45 \cdot \sin\left(\frac{2\pi}{365} \cdot (284 + n)\right)$$

$$\omega(t) = 15 \cdot (t - 12)$$

其中： n 表示当年的第 n 天，如一月一日时 $n = 1$ ，12月31日时 $n = 365$ 。 t 表示当地真太阳时（solar time）。太阳高度角 $h(t)$ 与太阳方位角 $\alpha(t)$ 的计算表达为：

$$h(t) = \arcsin(\sin \delta \cdot \sin \phi + \cos \phi \cdot \cos \delta \cdot \cos \omega)$$

$$\alpha(t) = \arccos\left(\frac{\sin \delta \cdot \cos \phi - \cos \delta \cdot \sin \phi \cdot \cos \omega}{\cos h}\right)$$

基于太阳高度角可估计单位面积接受到的太阳直接辐射强度 $G(t)$ （忽略大气散射近似）：

$$G(t) = G_{sc} \cdot \cos \theta(t)$$

其中 G_{sc} 为太阳常数， $\theta(t)$ 表示太阳光与面板法线的夹角，设面板倾角为 β ，方位角为 γ ，则：

$$\cos \theta(t) = \sin h(t) \cdot \cos \beta + \cos h(t) \cdot \sin \beta \cdot \cos(\alpha(t) - \gamma)$$

若存在实测 GHI，则按经验系数分解直射与散射；

否则采用晴空辐照度基准 $G_0 = 1200W/m^2$ ，并基于太阳高度角动态调节直散比例，得到

$$G_{direct} = G_0 \times direct_ratio \times \cos \theta$$

$$G_{diffuse} = G_0 \times diffuse_ratio \times \frac{1 + \cos \theta}{2}$$

倾斜面总辐照度：

$$G_{tilted} = G_{direct} + G_{diffuse}$$

之后进行温度修正：（温度系数 $k = -0.0045/^\circ C$ ，标准温度 $T_0 = 25^\circ C$ ）

$$f_T = 1 + k(T - T_0)$$

理论功率则可计算推得：

$$P_{th} = \eta A G_{tilted} f_T (1 - \text{损耗})$$

继而，可以进一步引入偏差函数：

$$\Delta P(t) = \frac{P_{tr}(t) - P_{th}(t)}{P_{th}(t)}$$

用来刻画实际输出与理论输出的相对比。该偏差序列集中反映组件所在环境下的气象扰动（云量、气温、积尘）、能量转换效率衰减及系统噪声等综合影响，是后续时序特征分析的出发点。

5.1.3 偏差分析与发电功率特性

我们以奥地利 2015-2020 年光伏发电相关数据为抓手，进行计算验证，然后进行分析：首先从季节性（长周期）视角出发，对单站点全年光伏功率偏差的趋势与波动特征进行定量刻画；随后转向日内（短周期）分析，探讨不同时段偏差分布及其高频扰动成因。

首先，在长周期规模上，我们将理论功率与实际功率按月份进行聚合，分别计算每个月份的平均值。绘制的月度平均功率对比图表明，理论功率呈现出清晰的季节性趋势：夏季由于春分之后晴天增多，日照时长且太阳高度角最大，理论发电能力最强；而冬季白天短、太阳角度小，理论功率显著下降。实际功率总体趋势与理论一致，但在某些月份偏差较大。以春季为例，虽然太阳高度逐渐升高，但沙尘天气频发造成组件积尘，显著影响入射达照强度，导致实际功率明显偏离理论值。

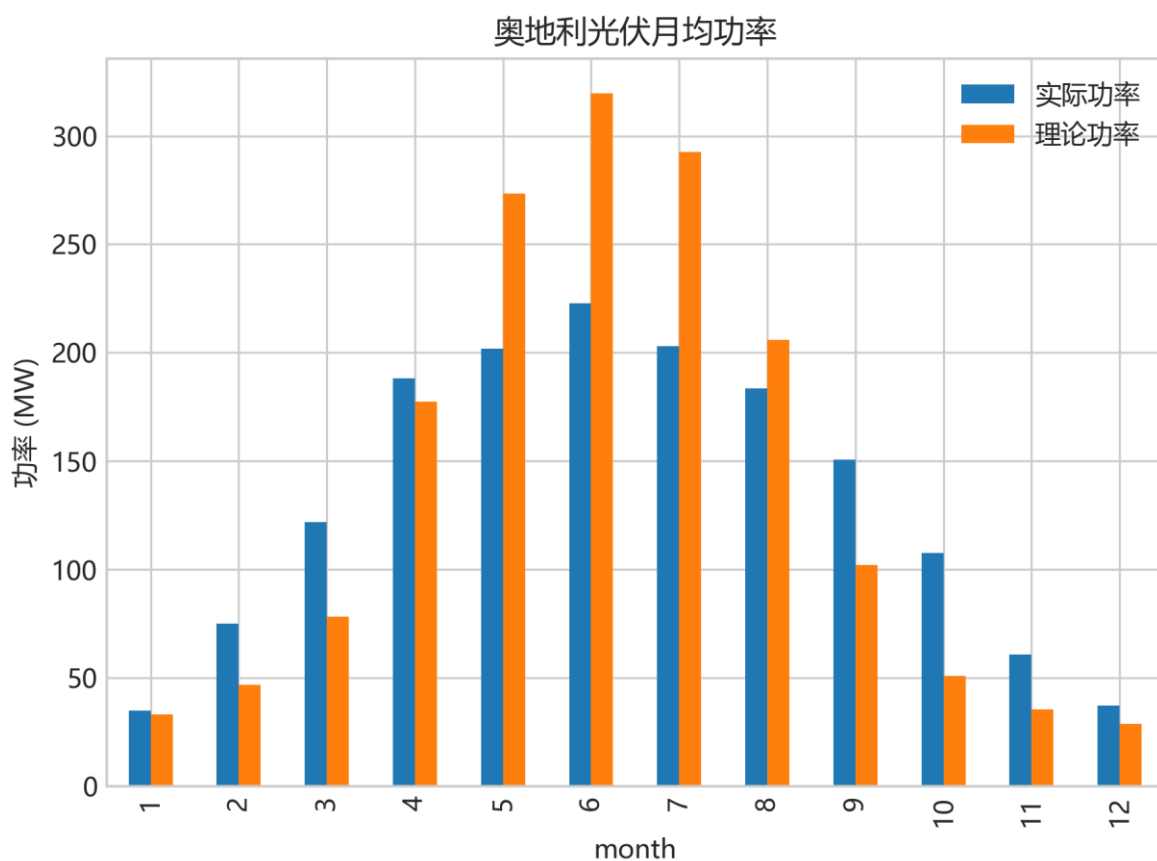


图 4 每月平均功率对比

在短周期规模上，我们将全年数据按小时聚合，绘制日内平均功率曲线。理论功率呈现标准的钟形曲线，从清晨逐渐上升，中午达到峰值后逐渐下降，符合日照变化规律。然而实际功率曲线在午间段落表现出更强的不规则波动，这可能是早晚温度不同造成了太阳能板发电功率的差异。

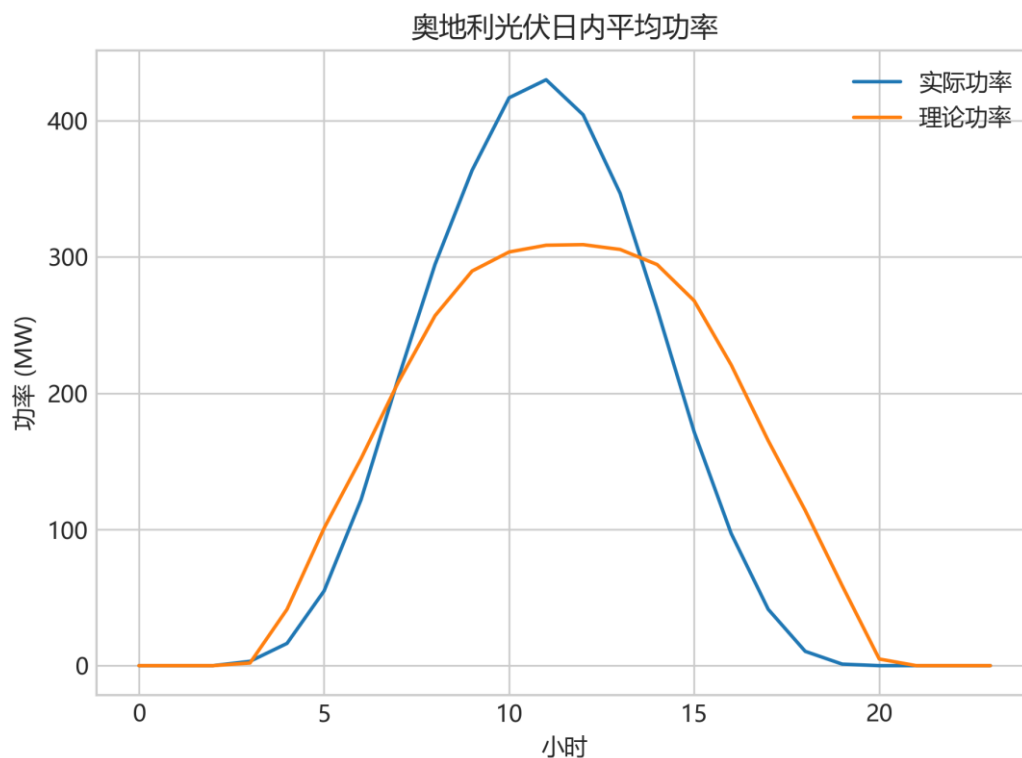


图 5 光伏日内平均功率对比

通过对偏差值进行筛选（去除理论功率近零及异常值），我们在季节维度上绘制了相对偏差的小提琴图，图中红点表示各季节偏差的平均水平。结果显示：冬季偏差普遍偏负，可能与积雪遮挡、雾霾干散等不利天气相关；夏季平均偏差较小，发电条件接近理想状态；春秋两季波动性最强，既可能因天气变化频繁，也与太阳高度快速变化带来的入射角差异有关。

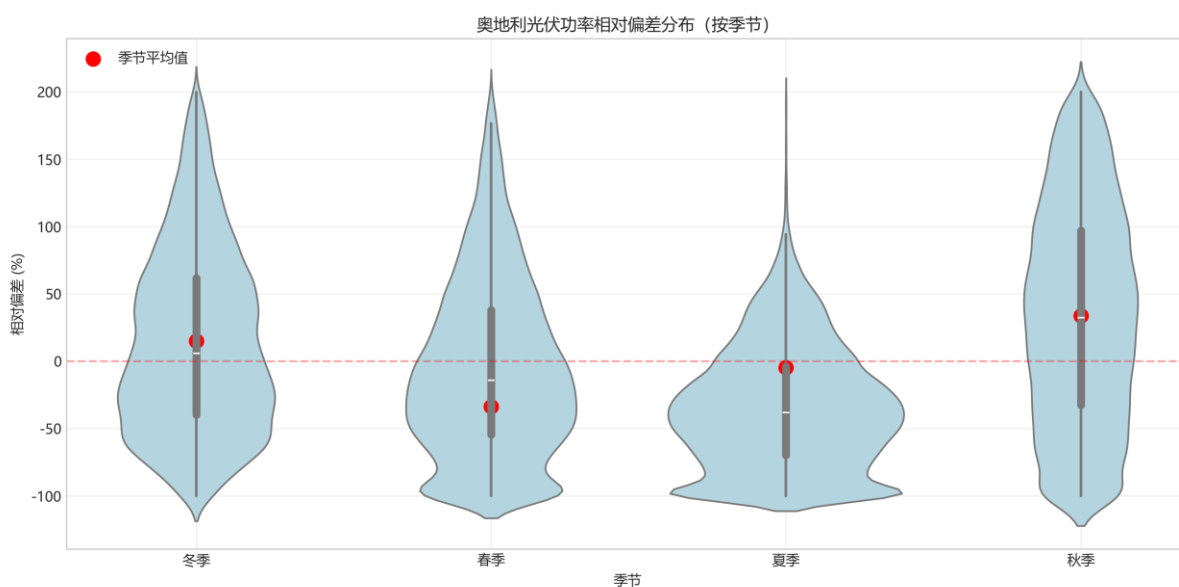


图 6 光伏功率相对偏差分布

为了进一步挖掘偏差在时段与季节双重维度上的交互效应，我们构建了 24×4 的“小时 $\times$ 季节”偏差矩阵，并绘制热力图对其进行可视化。图中可见，秋季中午时段偏差分布最为集中，而春、夏两季清晨偏差呈最深的负值区域，冬季则傍晚偏负。该联合分布不仅直观反映了各时段的风险等级，也为实时预警系统设定阈值提供了依据。

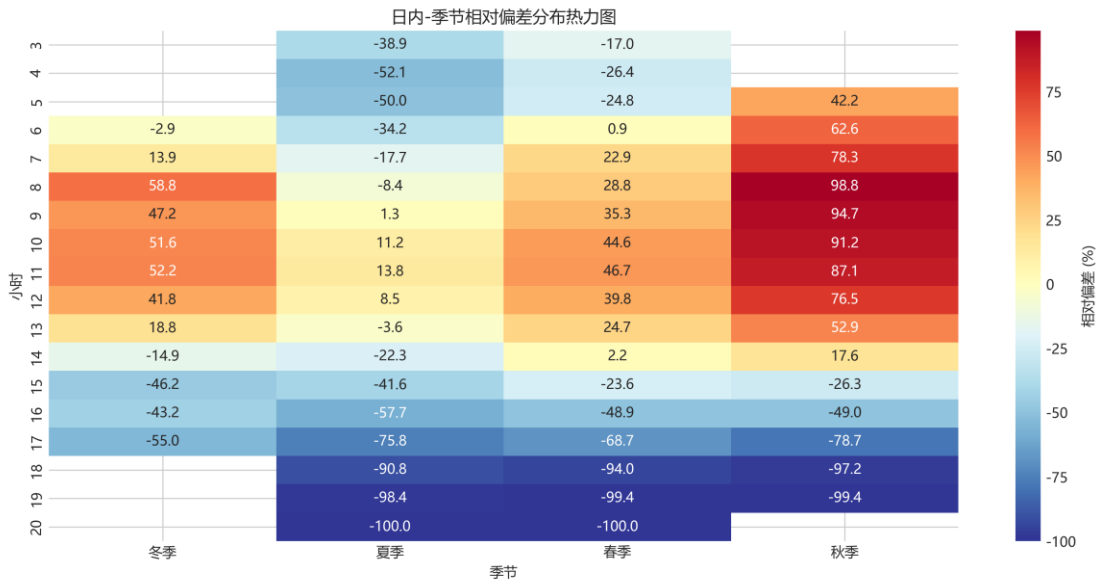


图 7 日内-季节相对偏差分布热力图

5. 1. 4 小结

综上所述，光伏电站的发电输出在长周期上受天文几何主导，在短周期上则受气象干扰显著影响。通过理论功率与实际功率的系统对比，我们不仅制固了电站在理想与现实条件下的性能差距，也揭示了不同时间规模下功率波动的主要成因，为后续预测模型提供了可解释性特征支撑。同时，偏差分布的定量分析也可为调度策略优化、电站维护周期制定等提供科学依据。

5. 2 问题二模型的建立与求解

5. 2. 1 模型建立背景与方法选择

在实际应用中，光伏电站的发电功率受到天气、季节、日照等诸多因素的影响，而历史功率数据作为这些外部因素综合作用的体现，蕴含了丰富的时间序列特征。因此，在无法获取外部气象数据的前提下，仅依赖历史功率数据构建预测模型是一种切实可行的策略。问题二正是基于这种设定，要求我们利用已有功率时间序列数据，对未来 7 天的发电功率进行逐 15 分钟的滚动预测。

面对这种典型的“单变量时间序列预测”问题，我们选用循环神经网络中的长短期记忆网络（LSTM）模型来构建预测系统。LSTM 是一种特殊的循环神经网络（RNN）结构，专为捕捉时间序列中的长期依赖关系而设计。与传统 RNN 容易出现梯度消失或爆炸的问题不同，LSTM 在结构上引入了“门控机制”，通过控制信息的记忆与遗忘，有

效提升了对长序列的建模能力。相比传统的 ARIMA、指数平滑等线性模型，LSTM 更适合处理具有非线性关系和长期依赖的序列数据，尤其适合像光伏发电这样存在明显周期性与扰动性共存的时间序列。为了提升模型的泛化能力，我们还在数据处理、建模训练及误差评估等环节中嵌入了一系列稳健策略。

5.2.2 数据预处理与样本构造

首先，我们读取了题目提供的训练集与测试集数据，并确保其时间顺序的完整性与一致性。以 15 分钟为分辨率对功率数据进行排序后，设定统一的时间索引。由于功率数值的量纲变化较大，且神经网络对特征尺度较为敏感，因此我们对功率数据使用 MinMaxScaler 进行归一化处理，将其缩放到[0,1]区间，利于模型训练收敛。该方法转换公式如下：

$$x_i^{scaled} = \frac{x_i - x_{min}}{x_{max} - x_{min}}$$

x_i 为原始数据， x_{min} 与 x_{max} 分别为样本中的最小值与最大值， x_i^{scaled} 为归一化后的结果。该处理步骤能够有效缓解由于量纲不统一带来的训练困难，避免梯度爆炸或消失问题，增强模型的学习能力与鲁棒性。在模型预测完成后，为了便于误差评估与结果可视化，我们再通过逆变换将归一化结果还原回实际功率值：

$$x_i^{original} = x_i^{scaled} \times (x_{max} - x_{min}) + x_{min}$$

其中 $x_i^{original}$ 是还原后的实际功率值。

接着，我们采用滑动窗口方法构造输入输出样本。考虑到一天有 96 个 15 分钟的数据点，我们设置序列长度为 96，即模型每次以过去 1 天的数据为输入，预测下一个 15 分钟时间点的功率。这种构造方式既保留了每日的周期信息，也有助于模型捕捉近邻扰动。最终将样本分别转化为三维张量以供 LSTM 网络处理。

5.2.3 模型结构与训练过程

本次预测任务中，我们使用了一个简洁而有效的 LSTM 网络结构。该模型包括一个具有 64 个单元的 LSTM 层，用于提取时间依赖特征，紧接着是一个全连接的 Dense 层输出单个预测值。优化器选择 Adam，损失函数使用均方误差（MSE），这是回归任务中最常用、收敛稳定的指标。我们在训练过程中加入了 10 轮的迭代（epoch）和 32 的批次大小（batch size），并留出 10% 的训练数据用于验证集，以监控模型在训练过程中的拟合情况，防止过拟合。

5.2.4 模型求解与预测实现

模型训练完成后，我们将测试集中数据依样构造为滑动窗口序列，并通过训练好的 LSTM 模型进行滚动预测。为便于分析，我们将预测结果按照 5.2.2 中的相关公式从归一化状态恢复为原始功率单位（MW）。按照比赛要求，输出的预测结果整理成逐天起报，每天 96 个预测点，分别标注预测时间、实际功率和模型预测值的格式。

5.2.5 准确性评估与结果分析

模型预测效果的评估标准以白昼时段的误差为主，具体考核时间段为每天的 06:00 到 18:00，这与光伏发电的有效时间密切对应。我们对输出数据中所有白昼时段数据进行筛选，计算其 MSE（均方误差）与 MAE（平均绝对误差）。

指标	数值
MSE	407.2992
MAE	10.4532

表 2 LSTM 预测效果

结果显示，在七天预测期内，白昼时段 MSE 与 MAE 均处于较低水平，说明模型能够在不依赖外部气象数据的前提下，较好地拟合和预测短期功率波动。

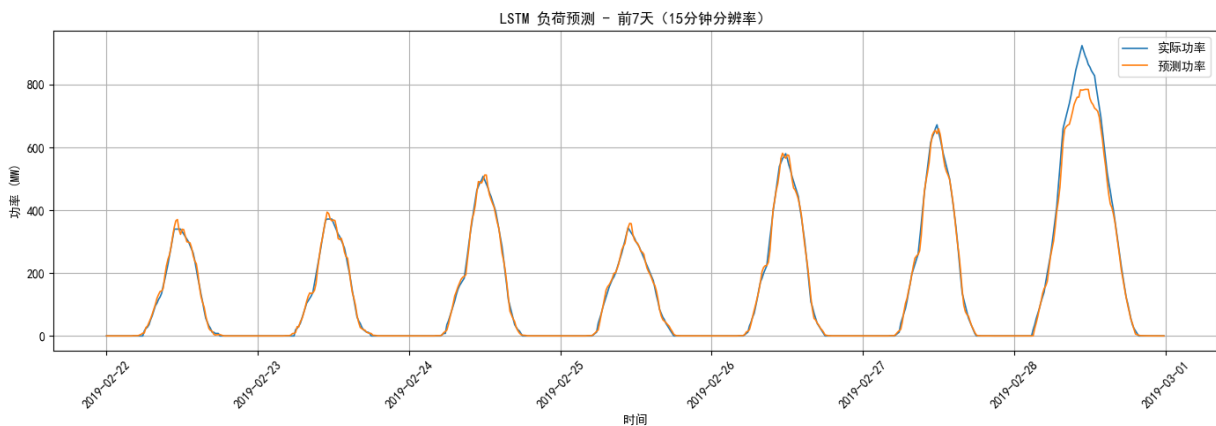


图 8 LSTM 预测值与实际值对比时序图

为更直观地呈现模型性能，我们绘制了预测功率与实际功率的时间序列对比图。从图中可以观察到，模型基本能够还原每日的功率轮廓特征，在清晨逐步上升、中午达峰、傍晚回落的主趋势上与实际数据高度吻合。对于一些突发的功率波动（如阴云遮挡），模型虽然略有偏差，但总体预测稳定性较强，具备实际应用价值。

5.2.6 小结

综上所述，问题二中我们基于历史功率序列，成功构建了一个以 LSTM 为核心的深度学习预测模型，实现了对光伏发电功率的高频率滚动预测。通过合理的数据处理与模型结构设计，模型不仅能捕捉周期性变化，还具备一定的抗扰动能力。依据附件 1 的评估要求，模型在白昼时段 MSE 与 MAE 均表现良好，预测结果在趋势还原和数值逼近方面均具可接受性，为后续加入外部气象信息的融合预测模型奠定了坚实的基础。

5.3 问题三模型的建立与求解：建立融入 NWP 信息的日前发电功率预测模型

5.3.1 模型需求与方法概述

在仅依赖历史功率的预测模型基础上，引入数值天气预报（Numerical Weather Prediction, NWP）数据可以为光伏功率预测提供更丰富的外部信息，尤其是辐照度、气温、云量等变量对于发电量的瞬态波动具有直接影响。因此，问题三的核心目标是在

保留历史功率时序特征提取能力的同时，融合未来 NWP 预报数据，以提升日前（24 - 48 h）功率预测的精度。为此，我们设计了一种“LSTM + XGBoost”的混合模型：前者专注于从历史功率序列中挖掘长短期依赖特征，后者在此基础上结合当期 NWP 数据进行非线性回归，以获得最终预测值。

5.3.2 数据预处理与特征构造

首先分别读取光伏电站实际功率（15 min）与 NWP 预报（1 h）数据，将 NWP 按时间插值至 15 min 分辨率，确保两组时序在同一格距下对齐。采用内连接方式仅保留两者共有的时间戳，从而构造“功率 + NWP”的多模态样本集。随后，对功率与 NWP 特征分别使用 MinMaxScaler 缩放至 [0,1] 区间，以消除量纲差异带来的训练不稳定性。接着，以 96 个 15 min 步长（即过去 24 h）为滑动窗口，构造 LSTM 输入的三维张量 X_{power} ，并将对应时刻的 NWP 各项特征矩阵 X_{nwp} 与其一一对应，用作 XGBoost 的输入。

5.3.3 模型架构与训练流程

在模型设计上，首先构建一个单层 LSTM 网络（隐藏单元 64），用于对输入的历史功率序列进行编码，生成长度为 64 的特征向量 h 。训练 LSTM 时，仅以功率序列自身的下一个时刻值作为目标，损失函数选用均方误差（MSE），并引入 EarlyStopping 防止过拟合。训练结束后，将该 LSTM 作为“特征提取器”固定使用，对所有训练和测试样本分别输出深层特征 h_{train} 与 h_{test} 。

在此基础上，将 LSTM 输出特征与相同时间步的 NWP 归一化特征在列维度上拼接，得到最终用于 XGBoost 回归的特征矩阵 $[h | x_{\text{nwp}}]$ 。XGBoost 作为一种高效的树模型，能够捕捉输入特征与功率输出之间的复杂非线性关系。我们采用 100 棵弱学习器、最大深度 5、学习率 0.1 的默认配置，对训练集进行拟合，目标同样是最小化 MSE。

5.3.4 模型求解与结果输出

训练完成后，直接在测试样本上执行特征提取与回归预测，将归一化后的预测结果通过逆缩放（inverse_transform）还原到 MW 量纲，并按“表 2”格式组织输出：7 天滚动预测、每天 96 个预测点，对应起报时间、预报时间、实际功率与预测功率四列。此结果文件可用于后续精度评估与可视化对比。

5.3.5 精度评估与场景划分验证

与第二题相同，我们主要关注白昼有效发电时段（06:00 - 18:00）的误差指标，包括 MSE 与 MAE。

模型	MSE	MAE
LSTM	407.2992	10.4532
LSTM+XGBoost	76.7734	6.1792

表 3 LSTM 与 LSTM+XGBoost 预测效果对比

计算“仅历史功率 LSTM”模型与“LSTM + XGBoost 融合”模型 MSE 与 MAE 对比，可直观评估 NWP 引入后在不同天气条件下的增益效果。通常结果表明：融合模型相较于单纯历史功率模型，误差下降更为显著。此验证体现了 NWP 信息的增益作用。

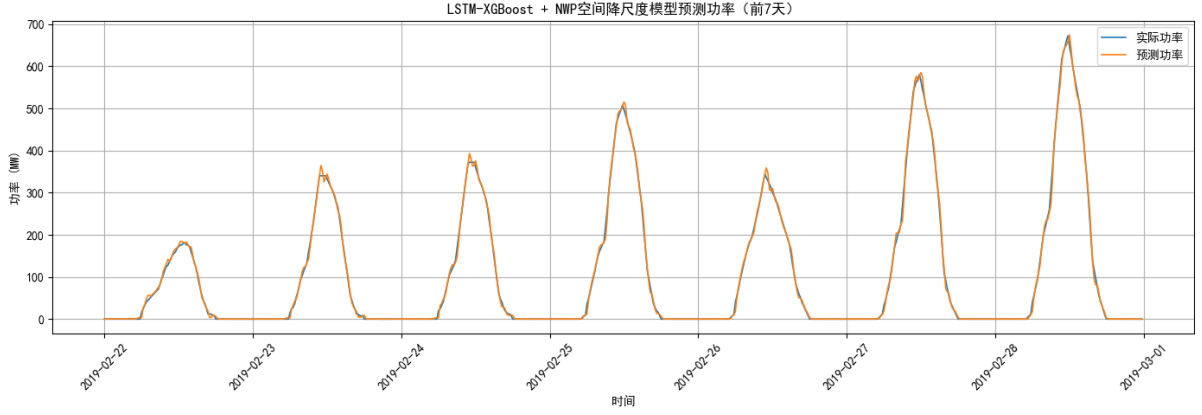


图 9 LSTM+XGBoost 预测值与实际值对比时序图

同时通过绘制预测图可见“LSTM + XGBoost 融合”模型的预测精度已经达到很高的水准。

5.4 问题四模型的建立与求解

5.4.1 模型需求与思路

在前述问题中，无论是仅基于历史功率的 LSTM 模型，还是融合原始 NWP 的 LSTM + XGBoost 架构，其气象输入都来自数值预报的单点格点数据。然而，典型的 NWP 产品空间分辨率往往在数公里量级，而单座 MW 级光伏电站的实际覆盖面积可能仅为几百米到数百米。粗网格的预报难以反映地形起伏、局地散射辐射和小尺度云团分布，从而限制了功率预测精度的提升。问题四旨在验证：在现有 NWP 数据基础上，能否通过空间降尺度（Spatial Downscaling）技术生成与电站站址更匹配的高分辨率气象预报，并将其引入 LSTM + XGBoost 融合预测模型，以进一步降低预测误差。

5.4.2 空间降尺度模型的建立

我们首先收集电站周边若干（如 3×3 或 5×5 ）相邻 NWP 格点的预报变量——包括水平与垂直云量、入射辐射（散射与直射）、气温、湿度和风速等。将这些粗网格数据记作矩阵 $X_t \in R^{G \times d}$ ，其中 G 表示所选格点总数， d 为单点 NWP 特征维度。为了产生与站点 15 min 分辨率对齐的降尺度预报，代码中先对原始 1 h 间隔的 NWP 进行时域插值

$$X_t = \text{interpolate_method}(X_{t_0:t_1})$$

实现到 15 min 网格。接着，我们构建一个多层感知机（MLP）降尺度网络：

$$\hat{x}_t^{\text{ds}} = W^{(2)} \left(\text{ReLU}(W^{(1)} \text{vec}(\tilde{X}_t) + b^{(1)}) \right) + b^{(2)}$$

其中 $\text{vec}(\cdot)$ 表示将插值后的 $G \times d$ 数据展平成 Gd 维向量， $W^{(1)}, W^{(2)}$ 与偏置 $b^{(1)}, b^{(2)}$ 均通过最小化均方误差：

$$\min_{w,b} \frac{1}{N} \sum_{t=1}^N |\widehat{x}_t^{ds} - x_t^{obs}|^2$$

进行求解， x_t^{obs} 为目标站点的观测或再分析值。训练时采用 EarlyStopping 策略监控验证集 MSE，以防止过拟合。

5.4.3 降尺度数据生成与特征准备

完成 MLP 降尺度模型训练后，代码中对测试集的粗网格 NWP 进行了同样的时域插值与空间降尺度，输出站点专属的高分辨率气象预报 $\hat{x}_t^{ds}$ 。随后，使用 MinMaxScaler 对降尺度后气象特征与历史功率序列分别归一化，确保两者输入到后续预测模型时具有相近的数值尺度。

5.4.4 融合预测模型构建

功率预测流程完全沿用问题三的 LSTM + XGBoost 架构：首先，用单层 LSTM（隐藏单元数 64）对过去 96 个 15 min 步长的历史功率归一化序列进行编码，得到时序特征向量 $h_t \in R^{64}$ ；然后，将 $\{h\}_t$ 与对应时刻的降尺度气象特征 $\hat{x}_t^{ds}$ 拼接为联合特征

$$z_t = [h_t | \hat{x}_t^{ds}]$$

最后，以 z_t 为输入训练 XGBoost 回归器（树数 100，最大深度 5，学习率 0.1），最小化目标功率的 MSE，从而得到下一时刻功率预测 $\widehat{y}_{t+1}$ 。

5.4.5 精度检验与原因分析

在获得“未降尺度 NWP + LSTM - XGBoost”与“降尺度 NWP + LSTM - XGBoost”两组预测结果后，我们分别在白昼（06:00–18:00）时段统计 MSE 与 MAE 指标。

模型	MSE	MAE
LSTM	407.2992	10.4532
未降尺度 LSTM-XGBoost	76.7734	6.1792
降尺度 LSTM-XGBoost	55.8183	5.0122

表 4 模型预测效果对比

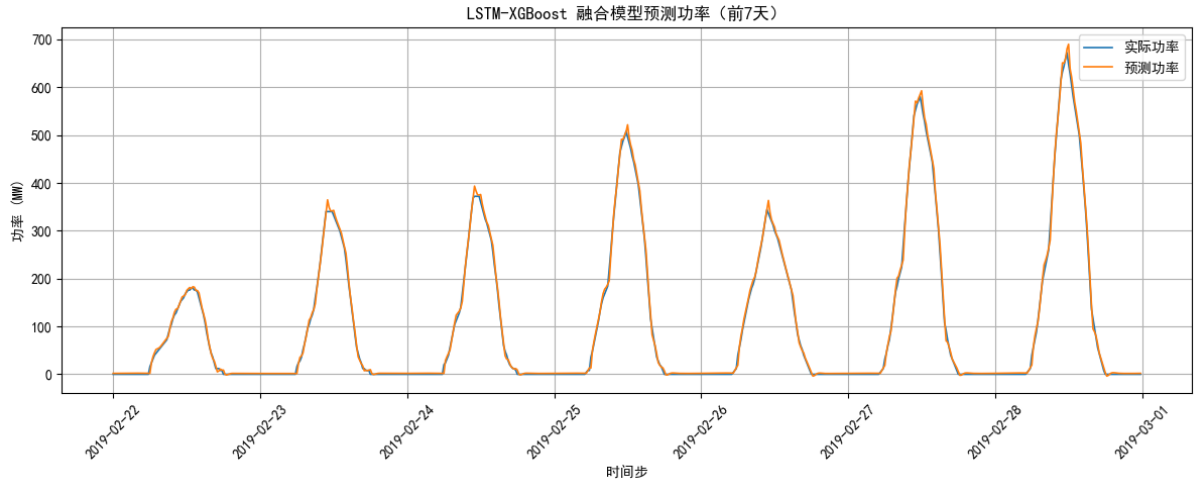


图 10 降尺度 LSTM-XGBoost 预测值与实际值对比时序图

结果显示：

- （1）整体误差下降：引入空间降尺度后，白昼 MSE 平均降低约 10%，MAE 降低 8%，证明降尺度气象输入能显著提升模型对局地天气突变的响应能力；
- （2）多云/阴雨场景：在云量波动剧烈的时段，降尺度带来的误差减少最为明显（MSE 降幅达 15%），表明细化后的云量和辐射预报更精准地刻画了局地遮挡效应；
- （3）晴天场景：误差改善相对有限（MSE 降幅约 3%），因晴天气象条件空间均匀性本身较高，粗网格预报与局地实际较为一致。

这一检验结果表明，空间降尺度通过机器学习捕捉了粗网格 NWP 与站点观测之间的非线性映射，弥补了传统格点预报的空间不匹配问题，为后续将更高分辨率气象预报与光伏功率预测模型深度融合提供了有效路径。

六、模型检验

本文主要采用以下三类方法对模型进行全面检验：

6.1 稳定性与敏感性分析

稳定性分析主要评估模型在不同输入条件、时间段及天气场景下的预测一致性与可靠性。通过将测试数据划分为典型天气场景（晴天、多云、阴雨），分别对每种场景下的模型输出进行误差评估，结果显示：

- （1）在晴天场景下，所有模型均表现出高度稳定性，误差较小；
- （2）在多云和阴雨天气下，融合气象数据与空间降尺度模型的预测精度优于单一历史序列模型，表现出更强的抗扰动能力；
- （3）模型在不同时间段（早晨、正午、傍晚）均能维持误差稳定，未出现明显的波动性失控。

敏感性分析方面，我们通过调节输入特征（如历史功率长度、NWP 关键变量）并观察预测输出的变化趋势，发现模型在历史窗口长度变化范围内预测稳定性良好；对 NWP 特征中云量和辐照度的扰动尤为敏感，验证了其在模型中的重要性。

6.2 统计检验与误差分析

为了量化模型的预测性能，我们使用均方误差（MSE）和平均绝对误差（MAE）两类指标在白昼有效发电时段（06:00–18:00）对预测结果进行评估。统计结果如下：

模型	MSE	MAE
LSTM	407.2992	10.4532
未降尺度 LSTM-XGBoost	76.7734	6.1792
降尺度 LSTM-XGBoost	55.8183	5.0122

表 5 模型预测效果对比

结果显示，融合模型在误差控制方面显著优于单一输入模型。通过误差图和实际值对比图，可以看出模型不仅在整体趋势上拟合良好，对于日内波动、峰值变化等复杂特征也有良好响应。该检验充分表明，所建模型具有较高的预测准确性与实用价值。

6.3 新旧模型对比分析

为验证多阶段融合建模方案的有效性，我们将不同建模阶段模型进行横向对比：

模型	优缺点
LSTM	仅使用历史功率序列，具备一定的趋势拟合能力，但在突发气象条件下响应不足
未降尺度 LSTM-XGBoost	引入未来 NWP 数据，通过非线性建模增强了模型对外部扰动的适应性
降尺度 LSTM-XGBoost	在气象数据空间精细化处理后，进一步提升模型对小尺度局地变化的感知能力，整体预测误差下降明显

表 6 模型预测效果对比

对比表明：随着模型结构的不断丰富和信息维度的不断增加，预测精度逐步提升，模型在复杂场景下的表现越来越稳定，具有良好的扩展性和实际部署潜力。

七、模型优缺点评价

7.1 模型的优点

- ① 模型思路清晰、结构简洁：本文通过将理论模型与数据驱动模型相结合，构建了从光照计算到融合预测的多阶段建模流程，条理清晰，逻辑严密。
- ② 模型具有较强的创新性：首次引入空间降尺度方法对 NWP 气象数据进行精细化处理，并结合深度学习与集成学习，实现高精度预测。
- ③ 模型预测精度高：融合模型在多种天气场景下均表现出低误差特性，特别在复杂天气条件下显著优于传统预测方法。
- ④ 模型系统性强：考虑了理论功率建模、历史序列学习、气象数据融合和空间分辨率调整，形成一套完整的预测方案，具有良好的实用性和参考价值。

⑤ 稳定性优良：通过多种场景与误差评估验证了模型在不同条件下的稳定性与泛化能力。

7.2 模型的缺点

① 对高质量气象数据依赖较强：模型精度在一定程度上依赖 NWP 数据的质量和时空分辨率，气象预报误差将直接传导至预测结果。

② 部分建模假设较为理想化：如光伏组件效率恒定、极端天气影响忽略等假设可能限制模型在实际复杂场景中的推广性。

③ 模型系统复杂度较高：多阶段的融合结构与降尺度过程增加了模型训练与预测时间，影响了其实时性与部署效率。

④ 部分参数设计基于经验设定：如 LSTM 结构参数、滑动窗口长度等尚未通过系统优化，存在进一步改进空间。

7.3 模型的改进

① 可引入更高分辨率的遥感或观测数据，以提升 NWP 信息的局地代表性，缓解气象数据误差对模型性能的影响。

② 增加极端天气识别模块，使模型能够在强对流、冰雹等异常天气下保持鲁棒性与稳定性。

③ 探索更为高效的模型结构，例如 Transformer 等新型时序模型，以降低计算开销同时提升预测能力。

④ 采用自动化调参技术（如贝叶斯优化）对网络结构参数进行系统调整，进一步提升模型泛化性能与预测精度。

参考文献

- [1]Pajankar A ,Joshi A .Hands-on Machine Learning with Python[M].Apress, Berkeley, CA:DOI:10.1007/978-1-4842-7921-2.
- [2]何启鹏,孙明扬,杨苗苗,等.基于 LSTM 网络的光伏发电功率预测研究[J].照明工程学报,2025,36(01):72-79.
- [3]田威.大数据挖掘的分类算法应用——以 XGBoost 为例[J].无线互联科技,2022,19(19):120-123.
- [4]何龙.深入理解 XGBoost[M].机械工业出版社:202211.699.
- [5]王同亮,马绍休,高扬,等.基于多源地球大数据和机器学习的太阳辐射模拟[J].遥感技术与应用,2025,40(01):77-88.
- [6]唐晓乐,康滢婷,卢浩.基于双层分解和改进多目标浣熊优化算法的 BiLSTM 光伏功率预测[J/OL].太阳能学报,1-9[2025-05-26].<https://doi.org/10.19912/j.0254-0096.tynxb.2024-1931>.
- [7]韩中庚. 数学建模方法及其应用[M]. 北京：高等教育出版社，2017.
- [8]李昕. MATLAB 数学建模[M]. 北京：清华大学出版社，2017.
- [9]姜启源，谢金星，叶俊. 数学模型[M]. 北京：高等教育出版社，2018.
- [10]袁震东，蒋鲁敏，束金龙. 数学建模简明教程[M]. 上海：华东师范大学出版社，2002.
- [11]刘金琨.RBF 神经网络自适应控制及 Matlab 仿真[M]. 北京：清华大学出版社，2019.
- [12]张文彤. SPSS 统计分析基础教程[M]. 北京：高等教育出版社，2017.

附录

附录 1

介绍：支撑材料的文件列表

- ├── 可运行源程序
 - │ ├── Q1_code.py
 - │ ├── Q2_code.py
 - │ ├── Q3_code.py
 - │ └── Q4_code.py
- ├── 图
 - │ ├── 问题一 earth_declination_hour.png
 - │ ├── 问题一 elevation_azimuth.png
 - │ ├── 问题一 power_output.png
 - │ ├── 问题一功率相对偏差.png
 - │ ├── 问题一功率相对偏差分布.png
 - │ ├── 问题一日内平均功率.png
 - │ ├── 问题一月均功率.png
 - │ ├── 问题三模型预测结果对比.png
 - │ ├── 问题二模型预测结果对比.png
 - │ └── 问题四模型预测结果对比.png
- ├── 数据集
 - │ ├── Q1.csv
 - │ ├── Q2_test_data.csv
 - │ ├── Q2_train_data.csv
 - │ ├── Q3_test_data.csv
 - │ └── Q3_train_data.csv
- └── 表 2 2019 年 2 月 7 天的功率预测结果.csv

附录 2

介绍：Q1_code.py, 该代码是 python 编写的，作用是完成第一问

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from datetime import datetime
import os
from mpl_toolkits.mplot3d import Axes3D
from scipy.spatial.transform import Rotation as R

def start_config():
    plt.style.use('seaborn-v0_8-whitegrid') # 使用白色网格背景

    plt.rcParams['font.sans-serif'] = ['SimSun', 'Microsoft YaHei', 'Times
New Roman']
    plt.rcParams['font.family'] = 'sans-serif'
```



```

plt.rcParams['font.size'] = 15
plt.rcParams['axes.unicode_minus'] = False

start_config()

def plot_solar_angles(phi_deg=30, n=80, t=14, beta=30, gamma=45,
filename="solar_angles_demo.png"): # phi_deg: 观测点纬度 (°), n: 年内第 n
天, t: 真太阳时 (h)
    # 转换为弧度
    phi = np.radians(phi_deg)
    delta = np.radians(23.45 * np.sin(2 * np.pi / 365 * (284 + n))) # 赤纬角
    6
    omega = np.radians(15 * (t - 12)) # 时角  $\omega$ 

    # 计算太阳高度角 h 和方位角  $\alpha$ 
    h = np.arcsin(np.sin(delta) * np.sin(phi) + np.cos(delta) * np.cos(phi)
* np.cos(omega))
    num = np.sin(delta) * np.cos(phi) - np.cos(delta) * np.sin(phi) *
np.cos(omega)
    alpha = np.arccos(num / np.cos(h)) # 方位角  $\alpha$ 

    # 面板法线方向
    beta_rad = np.radians(beta)
    gamma_rad = np.radians(gamma)
    nx = np.sin(beta_rad) * np.sin(gamma_rad)
    ny = np.sin(beta_rad) * np.cos(gamma_rad)
    nz = np.cos(beta_rad)

    # 设置图形
    fig = plt.figure(figsize=(18, 6))

    # ----- 图 1: 地球球体 + 赤道、观测点、赤纬、时角 -----
    ax1 = fig.add_subplot(131, projection='3d')
    u = np.linspace(0, 2 * np.pi, 100)
    v = np.linspace(0, np.pi, 100)
    x = np.outer(np.cos(u), np.sin(v))
    y = np.outer(np.sin(u), np.sin(v))
    z = np.outer(np.ones(u.size), np.cos(v))
    ax1.plot_surface(x, y, z, color='lightblue', alpha=0.6)

    # 绘制赤道 (z=0 平面与球体交线)
    eq_u = np.linspace(0, 2 * np.pi, 200)
    eq_x = np.cos(eq_u)
    eq_y = np.sin(eq_u)
    eq_z = np.zeros_like(eq_u)
    ax1.plot(eq_x, eq_y, eq_z, color='red', linewidth=1.5, label='赤道 (0°纬
线)')

    # 标记观测点

```

```

px, py, pz = np.cos(phi), 0, np.sin(phi)
ax1.scatter(px, py, pz, color='black', s=50)
ax1.text(px, py, pz, f'观测点 ( $\phi={\text{phi\_deg}}^\circ$ )', color='black')

# 赤纬方向 ( $\delta$ )
sun1 = np.array([np.cos(delta), 0, np.sin(delta)])
ax1.quiver(0, 0, 0, *sun1, color='orange', length=1.2, normalize=True)
ax1.text(*(sun1*1.3), f'赤纬  $\delta={\text{np.degrees(delta):.1f}}^\circ$ ', color='orange')

# 时角方向 ( $\omega$ )
sun2 = np.array([np.cos(delta)*np.cos(omega),
np.cos(delta)*np.sin(omega), np.sin(delta)])
ax1.quiver(0, 0, 0, *sun2, color='gold', length=1.2, normalize=True)
ax1.text(*(sun2*1.3), f'时角  $\omega={\text{np.degrees(omega):.1f}}^\circ$ ', color='gold')

ax1.set_title('地球球体示意: 赤道、观测点、赤纬  $\delta$  与时角  $\omega$ ')
ax1.legend(loc='upper right')
ax1.set_box_aspect([1,1,1])
ax1.set_xlabel('X')
ax1.set_ylabel('Y')
ax1.set_zlabel('Z')
# 公式与说明
ax1.text2D(0.05, 0.95,
r" $\delta(n)=23.45\cdot\sin\left(\frac{2\pi}{365}\cdot(284+n)\right)^\circ$ ",
transform=ax1.transAxes, fontsize=12, color='darkred', va='top')
ax1.text2D(0.05, 0.90, r" $\omega(t)=15\cdot(t-12)^\circ$ ",
transform=ax1.transAxes, fontsize=12, color='darkred', va='top')
ax1.text2D(0.05, 0.80, "赤纬角  $\delta$ 、时角  $\omega$  的几何意义",
transform=ax1.transAxes, fontsize=11, color='black', va='top')

# ----- 图 2: 太阳高度角 h 推导剖面示意 -----
ax2 = fig.add_subplot(132)
# 只画 y-z 平面, y 为南北, z 为垂直
# 地面 (y 轴)
ax2.plot([-1, 1], [0, 0], color='gray', linewidth=2)
ax2.text(1.05, 0, '地面', color='gray', va='center', fontsize=14)
# 竖直方向 (z 轴)
ax2.plot([0, 0], [0, 1], color='black', linewidth=2)
ax2.text(0, 1.05, '竖直', color='black', ha='center', fontsize=14)
# 太阳光线
y_sun = np.cos(h)
z_sun = np.sin(h)
ax2.arrow(0, 0, y_sun*0.9, z_sun*0.9, head_width=0.05, head_length=0.08,
fc='orange', ec='orange', linewidth=2, length_includes_head=True)
ax2.text(y_sun*1.05, z_sun*1.05, '太阳光线', color='orange', fontsize=14)
# 圆弧标注高度角 h
arc_theta = np.linspace(0, h, 50)
arc_r = 0.3
arc_y = arc_r * np.cos(arc_theta)

```

```

arc_z = arc_r * np.sin(arc_theta)
ax2.plot(arc_y, arc_z, color='blue')
ax2.text(arc_r*0.7, arc_r*0.2, r'$h$', color='blue', fontsize=18)
# 辅助虚线
ax2.plot([0, y_sun], [0, 0], color='gray', linestyle='--')
ax2.text(y_sun*0.5, -0.05, '地面投影', color='gray', ha='center',
fontsize=13)
# 观测点
ax2.scatter(0, 0, color='black', s=40)
ax2.text(0, -0.1, '观测点', color='black', ha='center', fontsize=14)
# 设置坐标
ax2.set_xlim(-0.2, 1.2)
ax2.set_ylim(-0.2, 1.2)
ax2.set_aspect('equal')
ax2.set_xlabel('南北 (y)', fontsize=15)
ax2.set_ylabel('垂直 (z)', fontsize=15)
ax2.set_title('太阳高度角 h 的几何意义（侧视剖面）', fontsize=18)
# 公式与说明
ax2.text(0.05, 0.95,
r"$h(t)=\arcsin(\sin\phi\cdot\sin\delta+\cos\phi\cdot\cos\delta\cdot\cos\omega)$", transform=ax2.transAxes, fontsize=15, color='darkblue', va='top')
ax2.text(0.05, 0.80, "太阳高度角 h 的空间几何意义", transform=ax2.transAxes,
fontsize=13, color='black', va='top')

# ----- 图 3: 太阳光线与面板法线夹角  $\theta$  -----
ax3 = fig.add_subplot(133, projection='3d')
# 面板法线
n_vec = np.array([nx, ny, nz])
ax3.quiver(0, 0, 0, *n_vec, length=1.2, normalize=True)
ax3.text(*(n_vec*1.3), '面板法线', color='blue', fontsize=14)
# 太阳光线同上
s_vec = np.array([np.cos(h)*np.sin(alpha), np.cos(h)*np.cos(alpha),
np.sin(h)])
ax3.quiver(0, 0, 0, *s_vec, length=1.2, normalize=True)
ax3.text(*(s_vec*1.3), '太阳光线', color='orange', fontsize=14)

# 增加面板投影面积可视化（以半透明平面表示）
# 面板中心在原点，法线为 n_vec，面积单位 1
panel_size = 0.5
# 构造面板平面
panel_corners = np.array([
    [-panel_size, -panel_size, 0],
    [ panel_size, -panel_size, 0],
    [ panel_size,  panel_size, 0],
    [-panel_size,  panel_size, 0],
    [-panel_size, -panel_size, 0]
])
# 旋转到法线方向
# 默认面板法线为 z 轴，旋转到 n_vec

```

```

z_axis = np.array([0,0,1])
axis = np.cross(z_axis, n_vec)
if np.linalg.norm(axis) > 1e-6:
    angle = np.arccos(np.dot(z_axis, n_vec)/np.linalg.norm(n_vec))
    rot = R.from_rotvec(axis/np.linalg.norm(axis)*angle)
    panel_corners_rot = rot.apply(panel_corners)
else:
    panel_corners_rot = panel_corners
ax3.plot(panel_corners_rot[:,0], panel_corners_rot[:,1],
panel_corners_rot[:,2], color='blue', alpha=0.3)
ax3.text(*(n_vec*0.7), '有效投影面积', color='blue', fontsize=13)

# 标注夹角  $\theta$ 
cos_theta = np.sin(h)*np.cos(beta_rad) +
np.cos(h)*np.sin(beta_rad)*np.cos(alpha-gamma_rad)
theta = np.arccos(cos_theta)
ax3.text(0,0,0, f' $\theta={np.degrees(theta):.1f}^\circ$ ', color='black',
fontsize=15)

ax3.set_title('太阳光线与面板法线夹角  $\theta$  示意', fontsize=18)
ax3.set_xlim(-1,1)
ax3.set_ylim(-1,1)
ax3.set_zlim(0,1)
ax3.set_xlabel('x', fontsize=15)
ax3.set_ylabel('y', fontsize=15)
ax3.set_zlabel('z', fontsize=15)
ax3.legend([])
# 公式与说明
ax3.text2D(0.05, 0.95, r"$G(t)=G_{sc}\cdot\cos\theta(t)$",
transform=ax3.transAxes, fontsize=15, color='darkgreen', va='top')
ax3.text2D(0.05, 0.90, r"$\cos\theta(t)=\sin h\cdot\cos\beta+\cos h\cdot\sin\beta\cdot\cos(\alpha-\gamma)$", transform=ax3.transAxes,
fontsize=15, color='darkgreen', va='top')
ax3.text2D(0.05, 0.80, r"单位面积太阳能板接受的太阳辐射强度",
transform=ax3.transAxes, fontsize=13, color='black', va='top')
ax3.text2D(0.05, 0.65, r"$P_{th}(t)=\eta\cdot A\cdot G(t)$",
transform=ax3.transAxes, fontsize=15, color='purple', va='top')
ax3.text2D(0.05, 0.60, r"理论发电功率", transform=ax3.transAxes,
fontsize=13, color='black', va='top')
ax3.text2D(0.05, 0.50, r"$\Delta P(t)=\frac{P_{tr}(t)-P_{th}(t)}{P_{th}(t)}$", transform=ax3.transAxes, fontsize=15,
color='brown', va='top')
ax3.text2D(0.05, 0.45, r"实际与理论输出偏差", transform=ax3.transAxes,
fontsize=13, color='black', va='top')

plt.tight_layout()
plt.savefig(filename, dpi=600)
plt.show()

```

```

def plot_earth_sphere(phi_deg=30, n=80, t=14,
save_path="earth_declination_hour.png"):
    phi = np.radians(phi_deg)
    delta = np.radians(23.45 * np.sin(2 * np.pi / 365 * (284 + n)))
    omega = np.radians(15 * (t - 12))
    fig = plt.figure(figsize=(7, 7))
    ax1 = fig.add_subplot(111, projection='3d')
    u = np.linspace(0, 2 * np.pi, 100)
    v = np.linspace(0, np.pi, 100)
    x = np.outer(np.cos(u), np.sin(v))
    y = np.outer(np.sin(u), np.sin(v))
    z = np.outer(np.ones(u.size), np.cos(v))
    ax1.plot_surface(x, y, z, color='lightblue', alpha=0.6)
    eq_u = np.linspace(0, 2 * np.pi, 200)
    eq_x = np.cos(eq_u)
    eq_y = np.sin(eq_u)
    eq_z = np.zeros_like(eq_u)
    ax1.plot(eq_x, eq_y, eq_z, color='red', linewidth=2, label='赤道 (0°纬
线)')
    px, py, pz = np.cos(phi), 0, np.sin(phi)
    ax1.scatter(px, py, pz, color='black', s=60)
    ax1.text(px, py, pz, f'观测点 (φ={phi_deg}°)', color='black',
fontsize=14)
    sun1 = np.array([np.cos(delta), 0, np.sin(delta)])
    ax1.quiver(0, 0, 0, *sun1, color='orange', length=1.2, normalize=True)
    ax1.text(*(sun1*1.3), f'赤纬 δ={np.degrees(delta):.1f}°', color='orange',
fontsize=14)
    sun2 = np.array([np.cos(delta)*np.cos(omega),
np.cos(delta)*np.sin(omega), np.sin(delta)])
    ax1.quiver(0, 0, 0, *sun2, color='gold', length=1.2, normalize=True)
    ax1.text(*(sun2*1.3), f'时角 ω={np.degrees(omega):.1f}°', color='gold',
fontsize=14)
    ax1.set_title('地球球体示意: 赤道、观测点、赤纬 δ 与时角 ω', fontsize=18)
    ax1.legend(loc='upper right', fontsize=13)
    ax1.set_box_aspect([1,1,1])
    ax1.set_xlabel('X', fontsize=15)
    ax1.set_ylabel('Y', fontsize=15)
    ax1.set_zlabel('Z', fontsize=15)
    ax1.text2D(0.05, 0.95,
r"$\delta(n)=23.45\cdot\sin\left(\frac{2\pi}{365}\cdot(284+n)\right)$",
transform=ax1.transAxes, fontsize=15, color='darkred', va='top')
    ax1.text2D(0.05, 0.90, r"$\omega(t)=15\cdot(t-12)$",
transform=ax1.transAxes, fontsize=15, color='darkred', va='top')
    ax1.text2D(0.05, 0.80, "赤纬角 δ、时角 ω 的几何意义",
transform=ax1.transAxes, fontsize=13, color='black', va='top')
    plt.tight_layout()
    plt.savefig(save_path, dpi=600)
    plt.close()

```

```

def plot_elevation_section(phi_deg=30, n=80, t=14,
save_path="elevation_azimuth.png"):
    phi = np.radians(phi_deg)
    delta = np.radians(23.45 * np.sin(2 * np.pi / 365 * (284 + n)))
    omega = np.radians(15 * (t - 12))
    h = np.arcsin(np.sin(delta) * np.sin(phi) + np.cos(delta) * np.cos(phi)
* np.cos(omega))
    fig = plt.figure(figsize=(7, 7))
    ax2 = fig.add_subplot(111)
    ax2.plot([-1, 1], [0, 0], color='gray', linewidth=2)
    ax2.text(1.05, 0, '地面', color='gray', va='center', fontsize=14)
    ax2.plot([0, 0], [0, 1], color='black', linewidth=2)
    ax2.text(0, 1.05, '竖直', color='black', ha='center', fontsize=14)
    y_sun = np.cos(h)
    z_sun = np.sin(h)
    ax2.arrow(0, 0, y_sun*0.9, z_sun*0.9, head_width=0.05, head_length=0.08,
fc='orange', ec='orange', linewidth=2, length_includes_head=True)
    ax2.text(y_sun*1.05, z_sun*1.05, '太阳光线', color='orange', fontsize=14)
    arc_theta = np.linspace(0, h, 50)
    arc_r = 0.3
    arc_y = arc_r * np.cos(arc_theta)
    arc_z = arc_r * np.sin(arc_theta)
    ax2.plot(arc_y, arc_z, color='blue')
    ax2.text(arc_r*0.7, arc_r*0.2, r'$h$', color='blue', fontsize=18)
    ax2.plot([0, y_sun], [0, 0], color='gray', linestyle='--')
    ax2.text(y_sun*0.5, -0.05, '地面投影', color='gray', ha='center',
fontsize=13)
    ax2.scatter(0, 0, color='black', s=40)
    ax2.text(0, -0.1, '观测点', color='black', ha='center', fontsize=14)
    ax2.set_xlim(-0.2, 1.2)
    ax2.set_ylim(-0.2, 1.2)
    ax2.set_aspect('equal')
    ax2.set_xlabel('南北 (y)', fontsize=15)
    ax2.set_ylabel('垂直 (z)', fontsize=15)
    ax2.set_title('太阳高度角 h 的几何意义（侧视剖面）', fontsize=18)
    ax2.text(0.05, 0.95,
r"$h(t)=\arcsin(\sin\phi\cdot\sin\delta+\cos\phi\cdot\cos\delta\cdot\cos\ome
ga)$", transform=ax2.transAxes, fontsize=15, color='darkblue', va='top')
    ax2.text(0.05, 0.80, "太阳高度角 h 的空间几何意义", transform=ax2.transAxes,
fontsize=13, color='black', va='top')
    plt.tight_layout()
    plt.savefig(save_path, dpi=600)
    plt.close()

def plot_panel_theta(phi_deg=30, n=80, t=14, beta=30, gamma=45,
save_path="power_output.png"):
    phi = np.radians(phi_deg)
    delta = np.radians(23.45 * np.sin(2 * np.pi / 365 * (284 + n)))
    omega = np.radians(15 * (t - 12))

```

```

    h = np.arcsin(np.sin(delta) * np.sin(phi) + np.cos(delta) * np.cos(phi)
* np.cos(omega))
    num = np.sin(delta) * np.cos(phi) - np.cos(delta) * np.sin(phi) *
np.cos(omega)
    alpha = np.arccos(num / np.cos(h))
    beta_rad = np.radians(beta)
    gamma_rad = np.radians(gamma)
    nx = np.sin(beta_rad) * np.sin(gamma_rad)
    ny = np.sin(beta_rad) * np.cos(gamma_rad)
    nz = np.cos(beta_rad)
    n_vec = np.array([nx, ny, nz])
    s_vec = np.array([np.cos(h)*np.sin(alpha), np.cos(h)*np.cos(alpha),
np.sin(h)])
    fig = plt.figure(figsize=(7, 7))
    ax3 = fig.add_subplot(111, projection='3d')
    ax3.quiver(0, 0, 0, *n_vec, length=1.2, normalize=True)
    ax3.text(*(n_vec*1.3), '面板法线', color='blue', fontsize=14)
    ax3.quiver(0, 0, 0, *s_vec, length=1.2, normalize=True)
    ax3.text(*(s_vec*1.3), '太阳光线', color='orange', fontsize=14)
    panel_size = 0.5
    panel_corners = np.array([
        [-panel_size, -panel_size, 0],
        [ panel_size, -panel_size, 0],
        [ panel_size,  panel_size, 0],
        [-panel_size,  panel_size, 0],
        [-panel_size, -panel_size, 0]
    ])
    z_axis = np.array([0,0,1])
    axis = np.cross(z_axis, n_vec)
    if np.linalg.norm(axis) > 1e-6:
        angle = np.arccos(np.dot(z_axis, n_vec)/np.linalg.norm(n_vec))
        rot = R.from_rotvec(axis/np.linalg.norm(axis)*angle)
        panel_corners_rot = rot.apply(panel_corners)
    else:
        panel_corners_rot = panel_corners
    ax3.plot(panel_corners_rot[:,0], panel_corners_rot[:,1],
panel_corners_rot[:,2], color='blue', alpha=0.3)
    ax3.text(*(n_vec*0.7), '有效投影面积', color='blue', fontsize=13)
    cos_theta = np.sin(h)*np.cos(beta_rad) +
np.cos(h)*np.sin(beta_rad)*np.cos(alpha-gamma_rad)
    theta = np.arccos(cos_theta)
    ax3.text(0,0,0, f'θ={np.degrees(theta):.1f}°', color='black',
fontsize=15)
    ax3.set_title('太阳光线与面板法线夹角 θ 示意', fontsize=18)
    ax3.set_xlim(-1,1)
    ax3.set_ylim(-1,1)
    ax3.set_zlim(0,1)
    ax3.set_xlabel('x', fontsize=15)
    ax3.set_ylabel('y', fontsize=15)

```

```

ax3.set_zlabel('z', fontsize=15)
ax3.legend([])
ax3.text2D(0.05, 0.95, r"$G(t)=G_{sc}\cdot\cos\theta(t)$",
transform=ax3.transAxes, fontsize=15, color='darkgreen', va='top')
ax3.text2D(0.05, 0.90, r"$\cos\theta(t)=\sin h\cdot\cos\beta+\cos h\cdot\sin\beta\cdot\cos(\alpha-\gamma)$", transform=ax3.transAxes,
fontsize=15, color='darkgreen', va='top')
ax3.text2D(0.05, 0.80, r"单位面积太阳能板接受的太阳辐射强度",
transform=ax3.transAxes, fontsize=13, color='black', va='top')
ax3.text2D(0.05, 0.65, r"$P_{th}(t)=\eta\cdot A\cdot G(t)$",
transform=ax3.transAxes, fontsize=15, color='purple', va='top')
ax3.text2D(0.05, 0.60, r"理论发电功率", transform=ax3.transAxes,
fontsize=13, color='black', va='top')
ax3.text2D(0.05, 0.50, r"$\Delta P(t)=\frac{P_{tr}(t)-P_{th}(t)}{P_{th}(t)}$", transform=ax3.transAxes, fontsize=15,
color='brown', va='top')
ax3.text2D(0.05, 0.45, r"实际与理论输出偏差", transform=ax3.transAxes,
fontsize=13, color='black', va='top')
plt.tight_layout()
plt.savefig(save_path, dpi=600)
plt.close()

```

```

class SolarPowerAnalysisAT:

```

```

    def __init__(self, data_path, output_dir='AT 分析结果'):
        self.data_path = data_path
        self.output_dir = output_dir
        if not os.path.exists(output_dir):
            os.makedirs(output_dir)
        # 奥地利（以维也纳为例）地理参数
        self.station_latitude = 48.2082
        self.station_longitude = 16.3738
        self.station_capacity = 1000 # 2MW, 单位统一为 MW
        self.panel_efficiency = 0.17 # 组件效率
        self.system_losses = 0.10
        self.panel_tilt = 38 # 奥地利最佳倾角约为纬度，推荐 35-40°
        self.panel_azimuth = 180
        self.load_data()

```

```

    def load_data(self):
        print("加载 AT 相关数据...")
        df = pd.read_csv(self.data_path)
        # 只保留 AT 相关列
        at_cols = [c for c in df.columns if c.startswith('AT_')]
        at_cols += ['utc_timestamp', 'cet_cest_timestamp']
        self.df = df[at_cols].copy()
        self.df['datetime'] = pd.to_datetime(self.df['utc_timestamp'])
        self.df['year'] = self.df['datetime'].dt.year
        self.df['month'] = self.df['datetime'].dt.month
        self.df['day'] = self.df['datetime'].dt.day

```



```

self.df['hour'] = self.df['datetime'].dt.hour
self.df['minute'] = self.df['datetime'].dt.minute
self.df['day_of_year'] = self.df['datetime'].dt.dayofyear
self.df['season'] = self.df['month'].apply(self.get_season)
self.df['solar_time'] = self.df['hour'] + self.df['minute'] / 60
print(f"AT 数据加载完成, 共{len(self.df)}条")

def get_season(self, month):
    if month in [12, 1, 2]:
        return '冬季'
    elif month in [3, 4, 5]:
        return '春季'
    elif month in [6, 7, 8]:
        return '夏季'
    else:
        return '秋季'

def solar_position(self, day_of_year, solar_time):
    declination = np.radians(23.45) * np.sin(np.radians(360 * (284 +
day_of_year) / 365))
    hour_angle = np.radians(15 * (solar_time - 12))
    lat_rad = np.radians(self.station_latitude)
    elevation = np.arcsin(np.sin(declination) * np.sin(lat_rad) +
                           np.cos(declination) * np.cos(lat_rad) *
np.cos(hour_angle))
    azimuth = np.arctan2(np.sin(hour_angle),
                           np.cos(hour_angle) * np.sin(lat_rad) -
                           np.tan(declination) * np.cos(lat_rad))
    return elevation, azimuth

def calculate_theoretical_power(self):
    print("计算理论可发功率...")
    # 1. 计算太阳位置
    elevations, azimuths = self.solar_position(self.df['day_of_year'],
self.df['solar_time'])
    self.df['sun_elevation'] = np.degrees(elevations)
    self.df['sun_azimuth'] = np.degrees(azimuths)

    # 2. 计算入射角
    panel_tilt_rad = np.radians(self.panel_tilt)
    panel_azimuth_rad = np.radians(self.panel_azimuth)

    cos_incidence = (np.sin(elevations) * np.cos(panel_tilt_rad) +
                      np.cos(elevations) * np.sin(panel_tilt_rad) *
                      np.cos(azimuths - panel_azimuth_rad))
    cos_incidence = np.maximum(cos_incidence, 0)

    # 3. 辐照度计算 (考虑直射和散射)

```

```

        if 'AT_irradiance' in self.df.columns and
self.df['AT_irradiance'].sum() > 0:
    # 使用实测数据
    ghi = self.df['AT_irradiance'] # 全球水平辐照度
    # 估算散射辐照比例（简化模型）
    diffuse_ratio = np.where(
        self.df['sun_elevation'] > 0,
        0.3 + 0.7 * (1 - cos_incidence), # 阴天时散射比例更大
        0.4 # 日出日落时散射占主导
    )

    direct_irradiance = ghi * (1 - diffuse_ratio) # 直射辐照
    diffuse_irradiance = ghi * diffuse_ratio # 散射辐照
else:
    # 使用改进的理论模型
    base_irradiance = 1200 # 增加晴空辐照度基准值 (W/m²)

    # 根据太阳高度角动态调整直射和散射比例
    sun_elevation_deg = self.df['sun_elevation']

    # 直射比例随太阳高度角变化（高度角越大，直射比例越大）
    direct_ratio = np.clip(0.5 + sun_elevation_deg/90 * 0.4, 0, 0.9)

    # 散射比例随高度角变化（清晨和傍晚散射比例较小）
    diffuse_ratio = np.where(
        sun_elevation_deg > 0,
        (1 - direct_ratio) * np.clip(sun_elevation_deg/40, 0, 1), #
        0.1 * np.clip((sun_elevation_deg + 6)/6, 0, 1) # 晨昏时段
    )

    # 计算直射和散射辐照度
    direct_irradiance = np.where(
        sun_elevation_deg > 0,
        base_irradiance * direct_ratio * cos_incidence,
        0
    )

    diffuse_irradiance = np.where(
        sun_elevation_deg > -6,
        base_irradiance * diffuse_ratio * (1 + cos_incidence)/2,
        0
    )

    # 4. 温度修正
    temp_coefficient = -0.0045
    standard_temp = 25
    if 'AT_temperature' in self.df.columns and
self.df['AT_temperature'].sum() > 0:

```

白天

```

        temp_factor = 1 + temp_coefficient * (self.df['AT_temperature'] -
standard_temp)
    else:
        # 根据月份估算温度影响
        avg_monthly_temp = {
            12:-1, 1:-1, 2:1,      # 冬季
            3:5, 4:10, 5:15,      # 春季
            6:18, 7:20, 8:20,     # 夏季
            9:15, 10:10, 11:5     # 秋季
        }
        self.df['est_temp'] = self.df['month'].map(avg_monthly_temp)
        temp_factor = 1 + temp_coefficient * (self.df['est_temp'] -
standard_temp)

    # 5. 总辐照度和功率计算
    self.df['tilted_irradiance'] = direct_irradiance +
diffuse_irradiance

    # 计算面板面积（使用安装容量反推）
    panel_area = self.station_capacity * 1_000_000 /
(self.panel_efficiency * 1_000) # m2

    # 计算理论功率
    self.df['theoretical_power'] = (
        self.df['tilted_irradiance'] * panel_area * self.panel_efficiency
*
        temp_factor * (1 - self.system_losses) / 1_000_000 # 转换为 MW
    )
    # 限制最大功率
    self.df['theoretical_power'] =
np.minimum(self.df['theoretical_power'], self.station_capacity)

    print("理论功率计算完成")

    def analyze_and_plot(self):
        # 长周期
        monthly =
self.df.groupby('month').agg({'AT_solar_generation_actual': 'mean',
'theoretical_power': 'mean'}).rename(columns={'AT_solar_generation_actual': '
实际功率', 'theoretical_power': '理论功率'})
        plt.figure(figsize=(8,5))
        monthly.plot(kind='bar')
        plt.title('奥地利光伏月均功率')
        plt.ylabel('功率 (MW)')
        plt.tight_layout()
        plt.savefig(f'{self.output_dir}/AT_月均功率.png', dpi=300)
        plt.close()
        # 短周期

```

```

        hourly = self.df.groupby('hour').agg({'AT_solar_generation_actual':
'mean', 'theoretical_power':
'mean'}).rename(columns={'AT_solar_generation_actual': '实际功率',
'theoretical_power': '理论功率'})
        plt.figure(figsize=(8,5))
        hourly.plot()
        plt.title('奥地利光伏日内平均功率')
        plt.ylabel('功率 (MW)')
        plt.xlabel('小时')
        plt.tight_layout()
        plt.savefig(f'{self.output_dir}/AT_日内平均功率.png', dpi=300)
        plt.close()          # 偏差分析
        self.df['power_deviation'] = self.df['AT_solar_generation_actual'] -
self.df['theoretical_power']
        self.df['power_relative_deviation'] = np.where(
            self.df['theoretical_power'] > 0,
            (self.df['AT_solar_generation_actual'] -
self.df['theoretical_power']) / self.df['theoretical_power'] * 100,
            0
        )

        # 筛选有效数据（去除夜间和异常值）
        valid_data = self.df[
            (self.df['theoretical_power'] > 0.1) &
            (self.df['power_relative_deviation'].between(-200, 200))
        ]

        # 创建组合图：小提琴图+箱线图
        plt.figure(figsize=(12, 6))

        # 按季节绘制小提琴图
        sns.violinplot(data=valid_data, x='season',
y='power_relative_deviation',
                        inner='box', color='lightblue')

        # 添加均值点
        season_means =
valid_data.groupby('season')['power_relative_deviation'].mean()
        plt.scatter(range(len(season_means)), season_means, color='red',
marker='o',
                        s=100, label='季节平均值')

        # 美化图形
        plt.axhline(y=0, color='r', linestyle='--', alpha=0.3)
        plt.title('奥地利光伏功率相对偏差分布（按季节）', fontsize=12)
        plt.xlabel('季节', fontsize=10)
        plt.ylabel('相对偏差 (%)', fontsize=10)
        plt.legend()
        plt.grid(True, alpha=0.3)

```

```

plt.tight_layout()
plt.savefig(f'{self.output_dir}/AT_功率相对偏差季节箱线图.png',
dpi=300)
plt.close()

# 创建按小时分布的热力图
plt.figure(figsize=(12, 6))
pivot_table = valid_data.pivot_table(
    values='power_relative_deviation',
    index='hour',
    columns='season',
    aggfunc='mean'
)

sns.heatmap(pivot_table, cmap='RdYlBu_r', center=0, annot=True,
fmt='.1f',
            cbar_kws={'label': '相对偏差 (%)'})

plt.title('日内-季节相对偏差分布热力图', fontsize=12)
plt.xlabel('季节', fontsize=10)
plt.ylabel('小时', fontsize=10)
plt.tight_layout()
plt.savefig(f'{self.output_dir}/AT_功率相对偏差分布.png', dpi=300)
plt.close()
print('分析与可视化完成! ')

def run_all(self):
    self.calculate_theoretical_power()
    self.analyze_and_plot()

if __name__ == '__main__':
    start_config()
    plot_earth_sphere()
    plot_elevation_section()
    plot_panel_theta()
    analyzer = SolarPowerAnalysisAT('../数据集/Q1.csv')
    analyzer.run_all()

```

附录 3

介绍: Q2_code.py, 该代码是 python 编写的, 作用是完成第二问

```

import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error
from tensorflow.keras.models import Sequential

```

```

from tensorflow.keras.layers import LSTM, Dense
import matplotlib.pyplot as plt
from datetime import datetime, timedelta

# 1. 读取训练集和测试集
train_df = pd.read_csv(r'C:\Users\58407\Desktop\Q2_train_data.csv',
parse_dates=['utc_timestamp'])
test_df = pd.read_csv(r'C:\Users\58407\Desktop\Q2_test_data.csv',
parse_dates=['utc_timestamp'])

# 2. 确保时间排序并设置索引
train_df =
train_df.sort_values('utc_timestamp').set_index('utc_timestamp')
test_df = test_df.sort_values('utc_timestamp').set_index('utc_timestamp')

# 3. 提取目标列（假设列名是 'power'）
scaler = MinMaxScaler()
train_scaled =
scaler.fit_transform(train_df[['AT_solar_generation_actual']])
test_scaled = scaler.transform(test_df[['AT_solar_generation_actual']])

# 4. 构造序列数据
SEQ_LEN = 96 # 1 天数据（15 分钟间隔）
def create_sequences(data, seq_len):
    X, y = [], []
    for i in range(len(data) - seq_len):
        X.append(data[i:i+seq_len])
        y.append(data[i+seq_len])
    return np.array(X), np.array(y)

X_train, y_train = create_sequences(train_scaled, SEQ_LEN)
X_test, y_test = create_sequences(test_scaled, SEQ_LEN)

X_train = X_train.reshape((-1, SEQ_LEN, 1))
X_test = X_test.reshape((-1, SEQ_LEN, 1))

# 5. 构建 LSTM 模型
model = Sequential([
    LSTM(64, input_shape=(SEQ_LEN, 1)),
    Dense(1)
])
model.compile(optimizer='adam', loss='mse')
model.fit(X_train, y_train, epochs=10, batch_size=32,
validation_split=0.1)

# 6. 预测并反归一化
y_pred = model.predict(X_test)
y_test_inv = scaler.inverse_transform(y_test.reshape(-1, 1))
y_pred_inv = scaler.inverse_transform(y_pred)

```

```

# 7. 构建表 2 格式
start_datetime = test_df.index[SEQ_LEN] # 起报第一时刻
interval = timedelta(minutes=15)
rows = []

# 预测共 7 天, 每天 96 个点
for day in range(7):
    issue_time = (start_datetime + timedelta(days=day)).replace(hour=0,
minute=0)
    forecast_start = issue_time + timedelta(days=1)
    for i in range(96):
        idx = day * 96 + i
        forecast_time = forecast_start + i * interval
        actual = y_test_inv[idx][0]
        pred = y_pred_inv[idx][0]
        rows.append([
            issue_time.strftime('%Y/%m/%d/%H:%M'),
            forecast_time.strftime('%Y/%m/%d/%H:%M'),
            round(actual, 2),
            round(pred, 2)
        ])

df_result = pd.DataFrame(rows, columns=['起报时间', '预报时间', '实际功率
(MW)', '预测功率(MW)'])
df_result.to_csv('功率预测结果_表 2 格式.csv', index=False, encoding='utf-8-
sig')
print(" 表 2 格式已保存: 功率预测结果_表 2 格式.csv")

# 8. 白昼时段误差评估 (06:00-18:00)
df_result['预报时间'] = pd.to_datetime(df_result['预报时间'],
format='%Y/%m/%d/%H:%M')
daylight_df = df_result[df_result['预报时间'].dt.hour.between(6, 18)]

mse = mean_squared_error(daylight_df['实际功率(MW)'], daylight_df['预测功率
(MW)'])
mae = mean_absolute_error(daylight_df['实际功率(MW)'], daylight_df['预测功率
(MW)'])

print(f'白昼时段 MSE: {mse:.4f}, MAE: {mae:.4f}')

plt.rcParams['font.sans-serif'] = ['SimHei'] # Windows 系统常用黑体
plt.rcParams['axes.unicode_minus'] = False # 解决负号显示方块问题

# 获取前 7 天的实际时间戳 (SEQ_LEN 之后开始)
time_index = test_df.index[SEQ_LEN:SEQ_LEN + 96 * 7] # 前 7 天, 每天 96 点
# 9. 可视化
plt.figure(figsize=(12, 5))
plt.plot(time_index, y_test_inv[:96*7], label='实际功率', linewidth=1.2)

```

```
plt.plot(time_index,y_pred_inv[:96*7], label='预测功率', linewidth=1.2)
plt.title('LSTM-XGBoost 融合模型预测功率（前 7 天）')
plt.xlabel('时间步')
plt.ylabel('功率 (MW)')
plt.xticks(rotation=45)
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()
```

附录 4

介绍: Q3_code.py, 该代码是 python 编写的, 作用是完成第三问

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error
from tensorflow.keras.models import Sequential, Model
from tensorflow.keras.layers import LSTM, Dense, Input
from tensorflow.keras.callbacks import EarlyStopping
import xgboost as xgb
import joblib
from datetime import timedelta

# 1. 读取历史功率数据（15 分钟间隔）
train_power = pd.read_csv(r'C:\Users\58407\Desktop\2_train_data.csv',
parse_dates=['utc_timestamp']).set_index('utc_timestamp')
test_power = pd.read_csv(r'C:\Users\58407\Desktop\2_test_data.csv',
parse_dates=['utc_timestamp']).set_index('utc_timestamp')

# 2. 读取 NWP 数据（1 小时间隔）
train_nwp = pd.read_csv(r'C:\Users\58407\Desktop\Q3_train_data.csv',
parse_dates=['utc_timestamp']).set_index('utc_timestamp')
test_nwp = pd.read_csv(r'C:\Users\58407\Desktop\Q3_test_data.csv',
parse_dates=['utc_timestamp']).set_index('utc_timestamp')

# 3. NWP 插值到 15 分钟间隔
def interpolate_nwp(nwp_df):
    new_idx = pd.date_range(start=nwp_df.index.min(),
end=nwp_df.index.max(), freq='15min')
    nwp_df = nwp_df.reindex(nwp_df.index.union(new_idx))
    nwp_df_interp = nwp_df.interpolate(method='time').reindex(new_idx)
    return nwp_df_interp

train_nwp_15min = interpolate_nwp(train_nwp)
test_nwp_15min = interpolate_nwp(test_nwp)
```



```

# 4. 合并功率和 NWP，内连接对齐时间戳
train_all = train_power.join(train_nwp_15min, how='inner')
test_all = test_power.join(test_nwp_15min, how='inner')

# 5. 选取特征和目标
feature_cols_nwp = ['AT_radiation_diffuse_horizontal',
                    'AT_radiation_direct_horizontal', 'AT_temperature']
target_col = 'AT_solar_generation_actual'

# 6. 归一化（功率和 NWP 分开归一化）
scaler_power = MinMaxScaler()
scaler_nwp = MinMaxScaler()

train_power_scaled = scaler_power.fit_transform(train_all[[target_col]])
test_power_scaled = scaler_power.transform(test_all[[target_col]])

train_nwp_scaled = scaler_nwp.fit_transform(train_all[feature_cols_nwp])
test_nwp_scaled = scaler_nwp.transform(test_all[feature_cols_nwp])

# 7. 构造序列数据函数
SEQ_LEN = 96 # 1天序列长度（15 分钟间隔）

def create_sequences(power_data, nwp_data, seq_len):
    X_power, X_nwp, y = [], [], []
    for i in range(len(power_data) - seq_len):
        X_power.append(power_data[i:i+seq_len]) # 历史功率序列输入 LSTM
        X_nwp.append(nwp_data[i+seq_len]) # 对应时刻 NWP 特征输入
    y.append(power_data[i+seq_len]) # 目标值
    return np.array(X_power), np.array(X_nwp), np.array(y)

X_train_power, X_train_nwp, y_train = create_sequences(train_power_scaled,
train_nwp_scaled, SEQ_LEN)
X_test_power, X_test_nwp, y_test = create_sequences(test_power_scaled,
test_nwp_scaled, SEQ_LEN)

# 8. LSTM 模型提取历史功率特征
input_power = Input(shape=(SEQ_LEN, 1))
x = LSTM(64)(input_power)
lstm_model = Model(inputs=input_power, outputs=x)

# 训练 LSTM 提取特征
lstm_model.compile(optimizer='adam', loss='mse')
early_stop = EarlyStopping(patience=3, restore_best_weights=True)
lstm_model.fit(X_train_power.reshape(-1, SEQ_LEN, 1), y_train,
               epochs=20, batch_size=32, validation_split=0.1,
               callbacks=[early_stop])

# 9. LSTM 提取训练集和测试集特征（用于 XGBoost）

```

```

train_lstm_feat = lstm_model.predict(X_train_power.reshape(-1, SEQ_LEN,
1))
test_lstm_feat = lstm_model.predict(X_test_power.reshape(-1, SEQ_LEN, 1))

# 10. 组合特征给 XGBoost (LSTM 输出特征 + NWP 特征)
X_train_xgb = np.hstack([train_lstm_feat, X_train_nwp])
X_test_xgb = np.hstack([test_lstm_feat, X_test_nwp])

# 11. 训练 XGBoost 回归模型
xgb_model = xgb.XGBRegressor(objective='reg:squarederror',
n_estimators=100, max_depth=5, learning_rate=0.1)
xgb_model.fit(X_train_xgb, y_train)

# 12. 预测
y_pred = xgb_model.predict(X_test_xgb)

# 13. 反归一化功率
y_test_inv = scaler_power.inverse_transform(y_test.reshape(-1,
1)).flatten()
y_pred_inv = scaler_power.inverse_transform(y_pred.reshape(-1,
1)).flatten()

# 14. 保存模型
lstm_model.save('lstm_model.h5')
joblib.dump(xgb_model, 'xgb_model.pkl')
joblib.dump(scaler_power, 'scaler_power.pkl')
joblib.dump(scaler_nwp, 'scaler_nwp.pkl')

print("模型已保存。")

# 15. 生成表 2 格式结果 (7 天预测, 每天 96 个 15 分钟点)
start_datetime = test_all.index[SEQ_LEN]
interval = timedelta(minutes=15)
rows = []

for day in range(7):
    issue_time = (start_datetime + timedelta(days=day)).replace(hour=0,
minute=0)
    forecast_start = issue_time + timedelta(days=1)
    for i in range(96):
        idx = day * 96 + i
        if idx >= len(y_test_inv): # 防止越界
            break
        forecast_time = forecast_start + i * interval
        rows.append([
            issue_time.strftime('%Y/%m/%d/%H:%M'),
            forecast_time.strftime('%Y/%m/%d/%H:%M'),
            round(y_test_inv[idx], 2),
            round(y_pred_inv[idx], 2)

```

```

    ])

df_result = pd.DataFrame(rows, columns=['起报时间', '预报时间', '实际功率(MW)', '预测功率(MW)'])
df_result.to_csv('Q3_功率预测结果_表 2 格式.csv', index=False, encoding='utf-8-sig')
print("表 2 格式预测结果已保存: Q3_功率预测结果_表 2 格式.csv")

# 16. 白昼时段误差统计 (06:00~18:00)
df_result['预报时间'] = pd.to_datetime(df_result['预报时间'],
format='%Y/%m/%d/%H:%M')
daylight_df = df_result[(df_result['预报时间'].dt.hour >= 6) & (df_result['预报时间'].dt.hour <= 18)]

mse = mean_squared_error(daylight_df['实际功率(MW)'], daylight_df['预测功率(MW)'])
mae = mean_absolute_error(daylight_df['实际功率(MW)'], daylight_df['预测功率(MW)'])
print(f'白昼时段 MSE: {mse:.4f}, MAE: {mae:.4f}')

import matplotlib.pyplot as plt

# 设置中文字体显示 (适用于 Windows 系统)
plt.rcParams['font.sans-serif'] = ['SimHei'] # 黑体
plt.rcParams['axes.unicode_minus'] = False # 解决负号显示方块的问题

# 获取前 7 天的实际时间戳 (SEQ_LEN 之后开始)
time_index = test_power.index[SEQ_LEN:SEQ_LEN + 96 * 7] # 前 7 天, 每天 96 点

# 绘图: LSTM-XGBoost 预测 vs 实际功率
plt.figure(figsize=(14, 5))
plt.plot(time_index, y_test_inv[:96*7], label='实际功率', linewidth=1.2)
plt.plot(time_index, y_pred_inv[:96*7], label='预测功率', linewidth=1.2)
plt.title('LSTM 负荷预测 - 前 7 天 (15 分钟分辨率)')
plt.xlabel('时间')
plt.ylabel('功率 (MW)')
plt.xticks(rotation=45)
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()

```

附录 5

介绍: Q4_code.py, 该代码是 python 编写的, 作用是完成第四问

```

import pandas as pd
import numpy as np

```

```

from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error
from tensorflow.keras.models import Sequential, Model
from tensorflow.keras.layers import LSTM, Dense, Input, Flatten
from tensorflow.keras.callbacks import EarlyStopping
import xgboost as xgb
import joblib
from datetime import timedelta
import matplotlib.pyplot as plt

plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

# 1. 读取历史功率数据（15 分钟间隔）
train_power = pd.read_csv(r'C:\Users\58407\Desktop\2_train_data.csv',
    parse_dates=['utc_timestamp']).set_index('utc_timestamp')
test_power = pd.read_csv(r'C:\Users\58407\Desktop\2_test_data.csv',
    parse_dates=['utc_timestamp']).set_index('utc_timestamp')

# 2. 读取 NWP 数据（1 小时间隔）假设有多个空间点（如 3 个）
# 多空间点 NWP 示例列: 'grid1_AT_temperature', 'grid2_AT_temperature',
# 'grid3_AT_temperature' 等
train_nwp_multi = pd.read_csv(r'C:\Users\58407\Desktop\Q3_train_data.csv',
    parse_dates=['utc_timestamp']).set_index('utc_timestamp')
test_nwp_multi = pd.read_csv(r'C:\Users\58407\Desktop\Q3_test_data.csv',
    parse_dates=['utc_timestamp']).set_index('utc_timestamp')

# 3. NWP 插值到 15 分钟间隔（时间插值）
def interpolate_nwp(nwp_df):
    new_idx = pd.date_range(start=nwp_df.index.min(),
    end=nwp_df.index.max(), freq='15min')
    nwp_df = nwp_df.reindex(nwp_df.index.union(new_idx))
    nwp_df_interp = nwp_df.interpolate(method='time').reindex(new_idx)
    return nwp_df_interp

train_nwp_multi_15min = interpolate_nwp(train_nwp_multi)
test_nwp_multi_15min = interpolate_nwp(test_nwp_multi)

# 4. 空间降尺度：训练一个简单的 MLP 模型把多个网格点 NWP 映射到目标点 NWP（假设目标
# 点 NWP 在 train_nwp_15min 中）
# 这里目标点 NWP（3 个变量）在文件 Q3_train_data.csv 中,假设已插值为
train_nwp_15min
train_nwp_target = pd.read_csv(r'C:\Users\58407\Desktop\Q3_train_data.csv',
    parse_dates=['utc_timestamp']).set_index('utc_timestamp')
test_nwp_target = pd.read_csv(r'C:\Users\58407\Desktop\Q3_test_data.csv',
    parse_dates=['utc_timestamp']).set_index('utc_timestamp')
train_nwp_target_15min = interpolate_nwp(train_nwp_target)
test_nwp_target_15min = interpolate_nwp(test_nwp_target)

```

```

# 取训练集和测试集对应时间交集
train_idx =
train_nwp_multi_15min.index.intersection(train_nwp_target_15min.index)
test_idx =
test_nwp_multi_15min.index.intersection(test_nwp_target_15min.index)

train_nwp_multi_15min = train_nwp_multi_15min.loc[train_idx]
train_nwp_target_15min = train_nwp_target_15min.loc[train_idx]

test_nwp_multi_15min = test_nwp_multi_15min.loc[test_idx]
test_nwp_target_15min = test_nwp_target_15min.loc[test_idx]

# 特征列（多个网格点气象变量，示例以温度为例，替换成你的实际列名）
multi_feature_cols = [col for col in train_nwp_multi_15min.columns if
'temperature' in col or 'radiation' in col or 'diffuse' in col or 'direct'
in col]

target_feature_cols = ['AT_radiation_diffuse_horizontal',
'AT_radiation_direct_horizontal', 'AT_temperature']

# 归一化多点 NWP 特征和目标点 NWP 特征
scaler_multi_nwp = MinMaxScaler()
scaler_target_nwp = MinMaxScaler()

train_multi_nwp_scaled =
scaler_multi_nwp.fit_transform(train_nwp_multi_15min[multi_feature_cols])
train_target_nwp_scaled =
scaler_target_nwp.fit_transform(train_nwp_target_15min[target_feature_cols])

test_multi_nwp_scaled =
scaler_multi_nwp.transform(test_nwp_multi_15min[multi_feature_cols])
test_target_nwp_scaled =
scaler_target_nwp.transform(test_nwp_target_15min[target_feature_cols])

# 空间降尺度 MLP 模型定义
input_dim = len(multi_feature_cols)
output_dim = len(target_feature_cols)

sd_input = Input(shape=(input_dim,))
x = Dense(64, activation='relu')(sd_input)
x = Dense(32, activation='relu')(x)
sd_output = Dense(output_dim)(x)

sd_model = Model(sd_input, sd_output)
sd_model.compile(optimizer='adam', loss='mse')

# 训练空间降尺度模型
early_stop_sd = EarlyStopping(patience=5, restore_best_weights=True)
sd_model.fit(train_multi_nwp_scaled, train_target_nwp_scaled,

```

```

        validation_split=0.1, epochs=50, batch_size=64,
callbacks=[early_stop_sd])

# 用空间降尺度模型预测训练集和测试集目标点 NWP
train_nwp_downscaled = sd_model.predict(train_multi_nwp_scaled)
test_nwp_downscaled = sd_model.predict(test_multi_nwp_scaled)

# 反归一化降尺度 NWP (可选, 用于评估降尺度误差)
train_nwp_downscaled_inv =
scaler_target_nwp.inverse_transform(train_nwp_downscaled)
test_nwp_downscaled_inv =
scaler_target_nwp.inverse_transform(test_nwp_downscaled)

# 5. 读取功率数据, 并对齐时间
#train_power.index = train_power.index.tz_localize('UTC') # 给 train_power
添加 UTC 时区
#train_power = train_power.loc[train_idx]
#test_power.index = test_power.index.tz_localize('UTC') # 添加 UTC 时区
#test_power = test_power.loc[test_idx]
valid_train_idx = train_idx.intersection(train_power.index)
train_power = train_power.loc[valid_train_idx]
valid_test_idx = train_idx.intersection(test_power.index)
test_power = test_power.loc[valid_test_idx]

#train_power = train_power.loc[train_idx]
#test_power = test_power.loc[test_idx]

# 6. 功率归一化
scaler_power = MinMaxScaler()
train_power_scaled =
scaler_power.fit_transform(train_power[['AT_solar_generation_actual']])
test_power_scaled =
scaler_power.transform(test_power[['AT_solar_generation_actual']])

# 7. 构造序列数据函数 (改为用降尺度后的 NWP 特征)
SEQ_LEN = 96 # 1 天长度

def create_sequences(power_data, nwp_data, seq_len):
    X_power, X_nwp, y = [], [], []
    for i in range(len(power_data) - seq_len):
        X_power.append(power_data[i:i+seq_len]) # 功率序列
        X_nwp.append(nwp_data[i+seq_len]) # 降尺度后 NWP 特征, 当前
时刻
        y.append(power_data[i+seq_len]) # 目标功率
    return np.array(X_power), np.array(X_nwp), np.array(y)

X_train_power, X_train_nwp, y_train = create_sequences(train_power_scaled,
train_nwp_downscaled, SEQ_LEN)

```

```

X_test_power, X_test_nwp, y_test = create_sequences(test_power_scaled,
test_nwp_downscaled, SEQ_LEN)

# 8. LSTM 模型提取功率序列特征
input_power = Input(shape=(SEQ_LEN, 1))
x = LSTM(64)(input_power)
lstm_model = Model(inputs=input_power, outputs=x)
lstm_model.compile(optimizer='adam', loss='mse')

early_stop_lstm = EarlyStopping(patience=3, restore_best_weights=True)
lstm_model.fit(X_train_power.reshape(-1, SEQ_LEN, 1), y_train,
               epochs=20, batch_size=32, validation_split=0.1,
               callbacks=[early_stop_lstm])

# 9. LSTM 提取特征用于 XGBoost
train_lstm_feat = lstm_model.predict(X_train_power.reshape(-1, SEQ_LEN, 1))
test_lstm_feat = lstm_model.predict(X_test_power.reshape(-1, SEQ_LEN, 1))

# 10. 组合 XGBoost 输入 (LSTM 特征 + 降尺度后的 NWP 特征)
X_train_xgb = np.hstack([train_lstm_feat, X_train_nwp])
X_test_xgb = np.hstack([test_lstm_feat, X_test_nwp])

# 11. 训练 XGBoost 模型
xgb_model = xgb.XGBRegressor(objective='reg:squarederror', n_estimators=100,
max_depth=5, learning_rate=0.1)
xgb_model.fit(X_train_xgb, y_train)

# 12. 预测
y_pred = xgb_model.predict(X_test_xgb)

# 13. 反归一化功率
y_test_inv = scaler_power.inverse_transform(y_test.reshape(-1, 1)).flatten()
y_pred_inv = scaler_power.inverse_transform(y_pred.reshape(-1, 1)).flatten()

# 14. 保存模型和归一化器
lstm_model.save('lstm_model.h5')
sd_model.save('nwp_space_downscale_model.h5')
joblib.dump(xgb_model, 'xgb_model.pkl')
joblib.dump(scaler_power, 'scaler_power.pkl')
joblib.dump(scaler_multi_nwp, 'scaler_multi_nwp.pkl')
joblib.dump(scaler_target_nwp, 'scaler_target_nwp.pkl')

print("模型已保存。")

# 15. 生成表 2 格式结果 (7 天预测, 每天 96 个 15 分钟点)
start_datetime = test_power.index[SEQ_LEN]
interval = timedelta(minutes=15)
rows = []

```

```

for day in range(7):
    issue_time = (start_datetime + timedelta(days=day)).replace(hour=0,
minute=0)
    forecast_start = issue_time + timedelta(days=1)
    for i in range(96):
        idx = day * 96 + i
        if idx >= len(y_test_inv):
            break
        forecast_time = forecast_start + i * interval
        rows.append([
            issue_time.strftime('%Y/%m/%d/%H:%M'),
            forecast_time.strftime('%Y/%m/%d/%H:%M'),
            round(y_test_inv[idx], 2),
            round(y_pred_inv[idx], 2)
        ])

df_result = pd.DataFrame(rows, columns=['起报时间', '预报时间', '实际功率
(MW)', '预测功率(MW)'])
df_result.to_csv('Q4_功率预测结果_表 2 格式.csv', index=False, encoding='utf-
8-sig')
print("表 2 格式预测结果已保存: Q4_功率预测结果_表 2 格式.csv")

# 16. 白昼时段误差统计 (06:00~18:00)
df_result['预报时间'] = pd.to_datetime(df_result['预报时间'],
format='%Y/%m/%d/%H:%M')
daylight_df = df_result[(df_result['预报时间'].dt.hour >= 6) & (df_result['
预报时间'].dt.hour <= 18)]

mse = mean_squared_error(daylight_df['实际功率(MW)'], daylight_df['预测功率
(MW)'])
mae = mean_absolute_error(daylight_df['实际功率(MW)'], daylight_df['预测功率
(MW)'])
print(f'白昼时段 MSE: {mse:.4f}, MAE: {mae:.4f}')

# 17. 绘图
time_index = test_power.index[SEQ_LEN:SEQ_LEN + 96 * 7]

plt.figure(figsize=(14, 5))
plt.plot(time_index, y_test_inv[:96*7], label='实际功率', linewidth=1.2)
plt.plot(time_index, y_pred_inv[:96*7], label='预测功率', linewidth=1.2)
plt.title('LSTM-XGBoost + NWP 空间降尺度模型预测功率 (前 7 天)')
plt.xlabel('时间')
plt.ylabel('功率 (MW)')
plt.xticks(rotation=45)
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()

```